



SỔ TAY THỰC HÀNH ECO2432 — BẢN SINH VIÊN 2026

Bản phát cho sinh viên. Các phần nội bộ, lời giải và dữ liệu phục vụ chấm bài đã được tách sang tài liệu giảng viên.

HỌC PHẦN: TIỀN ĐIỆN TỬ & HỢP ĐỒNG THÔNG MINH (ECO2432)

MỤC LỤC

Bản đồ codebase xuyên suốt	5
Kế hoạch hoàn thiện và trình bày	35
10 chủ đề dự án gợi ý	48
Lab 1 - CHUẨN BỊ MÔI TRƯỜNG LÀM VIỆC	7
Lab 2 - VÍ VÀ GIAO DỊCH ĐẦU TIÊN	9
Lab 3 - ĐỌC GIAO DỊCH VÀ HỢP ĐỒNG TRÊN ETHERSCAN	10
Lab 4 - NHẬN DIỆN HỢP ĐỒNG CÓ RỦI RO	12
Lab 5 - VIẾT ĐẶC TẢ CHO CÔNG CỤ PHÂN TÍCH DÒNG TIỀN	13
Lab 6 - SINH MÃ BẰNG AI VÀ KIỂM TRA KẾT QUẢ	15
Lab 7 - TÍNH CHI PHÍ VẬN HÀNH THỰC TẾ	17
Lab 8 - THIẾT KẾ QUY TẮC KINH TẾ CHO SẢN PHẨM	19
Lab 9 - HỢP ĐỒNG ĐẦU TIÊN: KẾT TIẾT KIỂM CÓ KHÓA THỜI GIAN	21
Lab 10 - RÀ SOÁT MÃ NGUỒN DO AI SINH RA	24
Lab 11 - CÀI QUY TẮC KINH TẾ VÀO SẢN PHẨM	26
Lab 12 - GATE REVIEW 1: DUYỆT CODEBASE VÀ PHẠM VI	29
Lab 13 - THỰC NGHIỆM MỘT VỤ MẤT TIỀN VÀ CÁCH KHẮC PHỤC	30
Lab 14 - RÀ SOÁT CHÉO GIỮA CÁC NHÓM	32
Lab 15 - GIAO DIỆN WEB VÀ ĐƯA SẢN PHẨM LÊN MẠNG	34

 PHẦN A: MÔN HỌC NÀY CHUẨN BỊ CHO CÔNG VIỆC NÀO?

Phần này cần nói thẳng với sinh viên ngay Buổi 1, vì nó quyết định các em học với thái độ nào.

A.1. Bối cảnh pháp lý Việt Nam đã thay đổi căn bản

Mốc	Nội dung
06/2025	Quốc hội thông qua Luật Công nghiệp Công nghệ số , lần đầu công nhận tài sản số là tài sản theo pháp luật Việt Nam
09/09/2025	Chính phủ ban hành Nghị quyết 05/2025/NQ-CP — chương trình thí điểm 5 năm cho thị trường tài sản mã hóa
01/01/2026	Luật Công nghiệp Công nghệ số có hiệu lực
03/2026	Bộ Tài chính ban hành Thông tư 15/2026/TT-BTC hướng dẫn nguyên tắc kế toán cho tổ chức tham gia thị trường tài sản mã hóa
04/2026	Thông tư 41/2026/TT-BTC có hiệu lực, hướng dẫn kê khai, khấu trừ, nộp và quyết toán thuế trên thị trường tài sản mã hóa

Ba con số cần nhớ: - **21,2 triệu** người trưởng thành Việt Nam đã từng dùng tài sản mã hóa (~17% dân số trưởng thành). - **Trên 100 tỷ USD** khối lượng giao dịch/năm (tương đương ~25% GDP), nhưng **gần như toàn bộ diễn ra trên sàn nước ngoài, ngoài phạm vi quản lý**. - **10.000 tỷ đồng** vốn điều lệ tối thiểu để được cấp phép vận hành sàn giao dịch tài sản mã hóa theo Nghị quyết 05.

A.2. Hệ quả nghề nghiệp — và đây là điều quan trọng nhất

Con số 10.000 tỷ đồng nghĩa là **sẽ chỉ có vài đơn vị được cấp phép**, và đều là tổ chức lớn. Vì vậy **hướng nghề nghiệp thực tế của sinh viên Kinh tế không phải là “đi làm lập trình viên hợp đồng thông minh”** — đó là việc của sinh viên CNTT, và cũng là việc AI đang làm thay nhiều nhất.

Hướng thực tế là: **dòng tiền 100 tỷ USD/năm đang chuyển từ ngoài luồng vào trong khuôn khổ pháp lý**, và quá trình đó cần rất nhiều người **hiểu cả tài chính lẫn dữ liệu on-chain**:

Vị trí việc làm	Việc cụ thể	Lab nào trong môn này chuẩn bị
Chuyên viên tuân thủ / phòng chống rửa tiền (AML/KYC) tại tổ chức cung ứng dịch vụ tài sản mã hóa	Truy vết dòng tiền ví, đối chiếu giao dịch, lập báo cáo giao dịch đáng ngờ	Lab 3, Lab 3B, Lab 4, Lab 6
Chuyên viên phân tích dữ liệu on-chain	Lấy dữ liệu qua API, phân tích dòng tiền vào/ra, dựng báo cáo	Lab 5, Lab 6, Lab 7
Kế toán / thuế tài sản số	Xác định giá trị, thời điểm ghi nhận, chi phí giao dịch	Lab 3, Lab 7
Chuyên viên phân tích nghiệp vụ (BA) sản phẩm tài sản số tại ngân hàng / công ty fintech	Viết đặc tả yêu cầu, làm việc với đội kỹ thuật, kiểm thử nghiệp vụ	Lab 5, Lab 8, Lab 10, Lab 11

Vị trí việc làm	Việc cụ thể	Lab nào trong môn này chuẩn bị
Chuyên viên thẩm định rủi ro dự án	Đọc hợp đồng thông minh, nhận diện điều khoản bất lợi cho người dùng	Lab 4, Lab 11, Lab 13, Lab 14

A.3. Điều thị trường tuyển dụng đòi hỏi ngay cả với người mới

Đối chiếu mô tả công việc trên các sàn tuyển dụng quốc tế và Việt Nam năm 2026:

- 1. **Bảo mật không còn là chuyên môn hẹp, nó là yêu cầu tối thiểu.** Ngay cả vị trí junior cũng được hỏi: giải thích lỗi tái nhập (reentrancy), nêu một sự cố mất tiền do lỗi phân quyền. → Đây là lý do Lab 10, Lab 13 và Lab 14 chiếm tỷ trọng lớn trong môn học.
- 2. **Đọc được dữ liệu on-chain** quan trọng hơn viết được hợp đồng, với người làm nghiệp vụ.
- 3. **Nhân lực blockchain tại Việt Nam thiếu nghiêm trọng**, nhưng thiếu ở nhóm *hiểu nghiệp vụ + đọc được dữ liệu*, không phải nhóm gõ được cú pháp.

Câu nói dành cho Buổi 1: “Môn này không đào tạo các em thành lập trình viên. Nó đào tạo các em thành người đọc được sổ sách của một hệ thống tài chính mới, và biết chỉ ra chỗ nào có vấn đề.”

🕒 PHẦN B: NGUYÊN TẮC CHUNG CỦA MỌI BÀI LAB

B.0. Cách đánh số bài lab

Buổi số mấy thì làm Lab số đó. Buổi 1 làm Lab 1, Buổi 9 làm Lab 9, hết Buổi 15 là hết Lab 15. Ngoại lệ duy nhất là **Lab 3B**, bài cá nhân 30 phút làm ở nhà sau Buổi 3; bài này không chiếm một buổi học và được nộp cùng `forensics.md`.

Tuần	Buổi	Lab
1	1 – 3	Lab 1 – 3
2	4 – 6	Lab 4 – 6
3	7 – 9	Lab 7 – 9
4	10 – 12	Lab 10 – 12 (Lab 12 là Gate Review 1 của repo nhóm)
5	13 – 15	Lab 13 – 15

B.1. Ba sản phẩm bắt buộc

Tệp	Viết khi nào	Nội dung
<code>SPEC.md</code>	Trước khi gõ prompt đầu tiên	Mô tả yêu cầu bằng tiếng Việt: đầu vào, đầu ra, quy tắc nghiệp vụ, trường hợp ngoại lệ
<code>AI_JOURNAL.md</code>	Trong lúc làm	Prompt đã dùng; chỗ AI làm sai và cách sinh viên sửa
Bảng chứng chạy được	Cuối buổi	Mã băm giao dịch (transaction hash), ảnh màn hình, hoặc đường dẫn công khai

Không có `SPEC.md` thì lab không được nghiệm thu, kể cả khi mã nguồn chạy đúng.

B.2. Mẫu `SPEC.md`

```
# SPEC – [Tên bài]
## 1. Mục đích
Một câu: hệ thống này giải quyết việc gì cho ai.

## 2. Đầu vào
– Dữ liệu / tham số gì, kiểu gì, ai cung cấp.

## 3. Quy tắc nghiệp vụ
– R1: ...
– R2: ...
(Mỗi quy tắc là một câu khẳng định kiểm tra được. Ví dụ: "Chỉ chủ sở hữu mới được rút tiền.")

## 4. Đầu ra
– Kết quả gì, hiển thị ở đâu.

## 5. Trường hợp ngoại lệ
– Nếu [tình huống] thì hệ thống phải [hành vi].
(Tối thiểu 3 trường hợp. Đây là phần phân biệt sinh viên khá và giỏi.)

## 6. Ngoài phạm vi
– Những gì bài này KHÔNG làm.
```

B.3. Mẫu `AI_JOURNAL.md`

```
# NHẬT KÝ LÀM VIỆC VỚI AI – [Tên bài]

## Lần 1
**Prompt:** (dán nguyên văn)
**AI trả về:** (tóm tắt)
**Đánh giá:** ✅ Đúng được / ⚠️ Phải sửa / ❌ Sai, bỏ
**Chỗ sai:** (mô tả cụ thể, kèm dòng mã)
**Cách sửa:** (sinh viên đã làm gì)
**Ai phát hiện:** AI tự nhận / Sinh viên phát hiện ← trường quan trọng nhất khi chấm
```

B.4. Tập `AGENTS.md` dùng chung cả học kỳ

Đặt ở thư mục gốc kho mã nguồn. Đây là chỉ dẫn thường trực cho công cụ AI, có tác dụng với Antigravity, Cursor, GitHub Copilot và các công cụ tương tự.

```
# AGENTS.md – Quy ước dự án ECO2432

## Ngôn ngữ và phiên bản
– Solidity `^0.8.20`. Không dùng cú pháp của phiên bản 0.7 trở về trước.
– Dùng thư viện OpenZeppelin Contracts **phiên bản 5.x**.
  Lưu ý: phiên bản 5 đã bỏ `_beforeTokenTransfer`, thay bằng `_update`.
– Python 3.10+.

## Quy tắc bắt buộc khi viết hợp đồng
1. Mọi hàm làm thay đổi trạng thái phải phát ra một `event`.
2. Mọi hàm chỉ dành cho chủ sở hữu phải có kiểm tra quyền rõ ràng.
3. Áp dụng thứ tự Checks – Effects – Interactions: kiểm tra điều kiện → cập nhật biến trạng thái
```

- chuyển tiền ra ngoài. Không được đảo thứ tự này.
- 4. Chuyển ETH bằng ``call{value: ...}("")`` kèm kiểm tra kết quả trả về. Không dùng ``transfer``.
- 5. Dùng ``error`` tùy biến thay cho chuỗi thông báo dài trong ``require``.
- 6. Không dùng ``tx.origin`` để xác thực.
- 7. Số tiền và tỷ lệ phần trăm dùng đơn vị điểm cơ bản (basis point, 1% = 100).

Quy tắc khi viết Python

- 1. Không ghi khóa API trực tiếp trong mã nguồn. Đọc từ biến môi trường.
- 2. Luôn kiểm tra mã trạng thái phản hồi trước khi xử lý dữ liệu.
- 3. Đơn vị wei phải đổi sang ETH trước khi hiển thị (chia cho 10^{18}).

Khi được yêu cầu sinh mã

- Giải thích ngắn gọn lựa chọn thiết kế trước khi đưa mã.
- Nếu yêu cầu chưa rõ, hỏi lại thay vì tự suy đoán.

B.5. Thang đánh giá dùng chung cho mọi lab

Mức	Tên	Điều kiện	Điểm
★	Đạt	Làm đúng theo hướng dẫn, có bằng chứng chạy được, giải thích được luồng chính	5,0 – 6,5
★★	Khá	Tự viết <code>SPEC.md</code> đầy đủ, bổ sung ít nhất 2 quy tắc nghiệp vụ ngoài đề bài, ghi nhận được ít nhất 1 lỗi của AI kèm bằng chứng	7,0 – 8,5
★★★	Giỏi	Có kiểm thử cho trường hợp gian lận, tìm và khắc phục được lỗi hổng thật, đo được chi phí trước và sau khi tối ưu, phản biện vững khi bị hỏi	9,0 – 10,0

B.6. Một codebase xuyên suốt học kỳ

Không tạo 15 kho mã nguồn rời. Học kỳ dùng đúng hai giai đoạn:

- Lab 1–7 : repo cá nhân – học công cụ và kỹ năng nền
- Lab 8 : lập nhóm, tạo repo nhóm từ `hce-web3-starter`
- Lab 9–15 : chỉ dùng repo nhóm này; mỗi lab thêm một phần vào cùng sản phẩm
- Tuần 6–7 : hoàn thiện, triển khai và bảo vệ chính repo nhóm đó

Từ Lab 8, mọi nhóm dùng cấu trúc thống nhất:

```
ten-du-an/
├── README.md           # giới thiệu sản phẩm và đường dẫn chạy thật
├── AGENTS.md           # quy ước cho công cụ AI
├── docs/
│   ├── PROJECT_PLAN.md # kế hoạch, vai trò, mốc công việc
│   ├── SPEC.md          # đặc tả nghiệp vụ đang có hiệu lực
│   ├── AI_JOURNAL.md    # nhật ký dùng AI và lỗi đã phát hiện
│   ├── ECONOMIC_RULES.md # quy tắc tiền, quyền lợi và quản trị
│   └── PRESENTATION_PLAN.md # kịch bản demo và phân công trình bày
├── contracts/
└── training/           # bài mẫu dùng để học kỹ thuật trong Lab 9, 10, 13
```

```
| └─ project/ProjectCore.sol
| └─ test/                  # ca kiểm thử của sản phẩm nhóm
| └─ web/index.html        # giao diện sản phẩm
| └─ evidence/lab-08...lab-15/ # ảnh, TxHash, link và biên bản từng lab
```

Mỗi lab từ Lab 8 phải kết thúc bằng một commit theo mẫu:

```
lab-XX: [kết quả chính của buổi]
```

Không xóa artifact của lab trước. Mỗi buổi là một lần nâng cấp cùng codebase; lịch sử commit là bằng chứng quá trình và được dùng khi vấn đáp.

LAB 1 — CHUẨN BỊ MÔI TRƯỜNG LÀM VIỆC

Buổi 1 · 75 phút · Cá nhân

Gắn với công việc

Mọi vị trí trong Phần A đều bắt đầu bằng việc có một ví và một môi trường làm việc sạch. Buổi này cũng là lần đầu sinh viên chạm vào công cụ lập trình có AI hỗ trợ — thứ đã thành mặc định trong nghề từ 2025.

Chuẩn đầu ra

Sinh viên có: (1) công cụ Antigravity chạy được, (2) ví MetaMask trên mạng thử nghiệm Sepolia, (3) kho mã nguồn cá nhân trên GitHub, (4) hiểu vai trò của tệp `AGENTS.md`.

Các bước

Bước 1 — Cài công cụ lập trình có AI hỗ trợ (20 phút) 1. Tải và cài **Antigravity**, đăng nhập bằng tài khoản Google. 2. Kiểm tra hoạt động: mở khung chat, gõ `Giải thích ngắn gọn: blockchain khác cơ sở dữ liệu thông thường ở điểm nào?` 3. Nếu máy không cài được, dùng phương án thay thế: **Gemini Code Assist** trong Visual Studio Code.

Bước 2 — Tạo ví MetaMask (20 phút) 1. Cài tiện ích MetaMask cho trình duyệt Chrome hoặc Edge. 2. Chọn *Create a new wallet*. 3. **Ghi 12 từ khôi phục (Secret Recovery Phrase) ra giấy**. Không chụp màn hình, không lưu vào ứng dụng ghi chú, không gửi qua tin nhắn. 4. Vào *Settings* → *Advanced* → *Show test networks* → bật. 5. Đổi mạng sang **Sepolia**.

Cần nhấn mạnh tại lớp: 12 từ này chính là quyền sở hữu tài sản. Không có tổng đài nào khôi phục được. Ví dùng trong môn học không chứa tiền thật, nhưng thói quen phải đúng ngay từ đầu.

Bước 3 — Nhận Sepolia ETH (10 phút) 1. Thử tại `https://cloud.google.com/application/web3/faucet/ethereum/sepolia`. 2. **Nếu bị từ chối** (rất hay xảy ra — các cổng cấp phát yêu cầu ví phải có số dư trên mạng chính để chống lạm dụng): đọc địa chỉ ví cho giảng viên, giảng viên chuyển từ ví kho bạc. 3. Điền địa chỉ ví vào bảng tính của lớp.

Bước 4 — Kho mã nguồn và tệp AGENTS.md (25 phút) 1. Tạo tài khoản GitHub (nếu chưa có), đặt tên tài khoản nghiêm túc — đây sẽ là hồ sơ nghề nghiệp. 2. Fork kho `hce-web3-starter` → sao chép về máy → mở bằng Antigravity. 3. Mở `AGENTS.md`, đọc từng dòng. Giảng viên giải thích: đây là bản quy ước nội bộ mà công cụ AI sẽ đọc và tuân theo trong mọi lần sinh mã. 4. Thêm vào cuối tệp một dòng của riêng mình, ví dụ: `- Chú thích trong mã viết bằng tiếng Việt không dấu.` 5. Lưu, rồi tạo bản ghi thay đổi đầu tiên: `bash git add . git commit -m "chore: thiet lap moi truong lam viec" git push`

Sản phẩm nộp

- Ảnh màn hình Antigravity đang mở kho mã nguồn.
- Đường dẫn tới bản ghi thay đổi đầu tiên trên GitHub.
- Địa chỉ ví đã điền vào bảng tính lớp.

Bẫy thường gặp

Bẫy	Xử lý
Sinh viên dùng chung một ví	Kiểm tra trùng địa chỉ trong bảng tính ngay cuối buổi

Bẫy	Xử lý
Cổng cấp phát từ chối	Ví kho bạc — đây là tình huống mặc định, không phải ngoại lệ
Chưa từng dùng Git	Chấp nhận thao tác qua giao diện GitHub Desktop hoặc nút commit trong Antigravity
Máy yếu, không cài được	Phương án dự phòng: Remix IDE trên trình duyệt + Gemini trên web

LAB 2 — VÍ VÀ GIAO DỊCH ĐẦU TIÊN

Buổi 2 · 75 phút · Cặp đôi

Gắn với công việc

Người làm tuân thủ hoặc kế toán tài sản số phải hiểu một giao dịch gồm những thành phần nào và tại sao nó thất bại. Không thể kiểm tra thứ mình chưa từng tự tay làm.

Chuẩn đầu ra

Sinh viên thực hiện được một giao dịch thành công và **một giao dịch thất bại có chủ đích**, giải thích được nguyên nhân thất bại.

Các bước

Bước 1 — Giao dịch thành công (20 phút) 1. Ghép cặp hai sinh viên ngồi cạnh nhau, trao đổi địa chỉ ví. 2. Chuyển **0,01 Sepolia ETH** cho bạn. 3. Quan sát trạng thái chuyển từ *Pending* sang *Confirmed*. Ghi lại mã băm giao dịch.

Bước 2 — Giao dịch thất bại có chủ đích (25 phút)

Chọn một trong hai tình huống:

Tình huống A — địa chỉ sai: sửa một ký tự trong địa chỉ người nhận. MetaMask sẽ từ chối vì sai mã kiểm tra (checksum). → Bài học: địa chỉ ví có cơ chế tự kiểm tra lỗi gõ nhầm, nhưng **không kiểm tra được người nhận có đúng ý bạn không**.

Tình huống B — không đủ phí: thử chuyển toàn bộ số dư (không chứa phí giao dịch). → Bài học: phí giao dịch (gas) luôn phải trả bằng đồng tiền gốc của mạng, không trừ vào số tiền chuyển.

Bước 3 — Ghi nhận (30 phút)

Sinh viên điền bảng sau vào `lab02.md`:

Trường	Giao dịch thành công	Giao dịch thất bại
Mã băm giao dịch		
Số tiền chuyển		
Phí giao dịch thực trả		
Trạng thái		
Nguyên nhân (nếu thất bại)		

Kèm một đoạn 3 câu trả lời: *Nếu bạn chuyển nhầm cho người lạ, có lấy lại được không? Vì sao?*

Sản phẩm nộp

`lab02.md` với 2 mã băm giao dịch và bảng đối chiếu.

LAB 3 — ĐỌC GIAO DỊCH VÀ HỢP ĐỒNG TRÊN ETHERSCAN

Buổi 3 · 75 phút · Cá nhân

Gắn với công việc

Đây là kỹ năng nền của chuyên viên tuân thủ và chuyên viên phân tích on-chain. Toàn bộ việc truy vết dòng tiền bắt đầu từ màn hình này.

Chuẩn đầu ra

Sinh viên đọc và giải thích được mọi trường trong một giao dịch; phân biệt được mã máy (bytecode) với mã nguồn đã xác thực; phân biệt hàm đọc với hàm ghi.

Các bước

Bước 1 — Mở xè giao dịch của chính mình (25 phút)

Dán mã băm giao dịch từ Lab 2 vào `https://sepolia.etherscan.io`. Giải thích từng trường:

Trường	Ý nghĩa	Vì sao người làm nghiệp vụ cần
Status	Thành công hay thất bại	Giao dịch thất bại vẫn mất phí — ảnh hưởng hạch toán
Block	Số thứ tự khối chứa giao dịch	Xác định thời điểm ghi nhận
Timestamp	Thời gian	Mốc ghi nhận doanh thu / chi phí
From / To	Ví gửi / ví nhận	Đối tượng cần xác minh danh tính
Value	Số tiền chuyển	Giá trị giao dịch
Transaction Fee	Phí thực trả	Chi phí giao dịch, cần hạch toán riêng
Gas Price	Đơn giá phí	Giải thích vì sao cùng một giao dịch mà phí khác nhau
Gas Limit	Mức gas tối đa ví gửi cho phép dùng	Đặt quá thấp → giao dịch thất bại vì hết gas (<i>Out of Gas</i>) nhưng vẫn mất phí
Gas Used	Lượng gas thực tế đã tiêu	Cùng Gas Price với Gas Used tính ra phí: Transaction Fee = Gas Used × Gas Price. Gas Used = Gas Limit là dấu hiệu hết gas
Nonce	Số thứ tự giao dịch của ví gửi	Phát hiện giao dịch bị bỏ sót hoặc thay thế

Bước 2 — Đọc một hợp đồng thật (30 phút)

Mở hợp đồng USDT hoặc USDC trên mạng chính tại `https://etherscan.io`.

Lưu ý về Proxy Contract. Khuyến nghị dùng **USDT** (`0xdAC17F958D2ee523a2206206994597C13D831ec7`) — các hàm nghiệp vụ hiện trực tiếp trong *Read/Write Contract*. **USDC** (`0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48`) là một *Proxy* (mẫu EIP-1967): tab *Read/Write Contract* chỉ hiện hàm quản trị proxy (`upgradeTo`, `changeAdmin`...). Muốn thấy hàm của token (`totalSupply`, `blacklist`...) phải chọn tab con **Read as Proxy / Write as Proxy**. Đây cũng là bài học: mã nguồn đã xác thực của proxy *không* phải logic thật — logic nằm ở địa chỉ implementation.

1. Tab **Contract**: phân biệt hai thứ

- *Bytecode*: chuỗi ký tự máy thực thi. Con người không đọc được.
- *Source Code (verified)*: mã nguồn do bên phát hành công bố và đã được đối chiếu khớp với bytecode.

Nếu một dự án không công bố mã nguồn đã xác thực, đó là dấu hiệu cần thận trọng.

2. Tab **Read Contract**: các hàm chỉ đọc dữ liệu, không tốn phí, không cần ví. Thử gọi `totalSupply()` và `balanceOf()` với một địa chỉ bất kỳ.

3. Tab **Write Contract**: các hàm thay đổi dữ liệu, tốn phí, phải ký bằng ví.

Bước 3 — Ghi nhận (20 phút)

Viết `forensics.md` gồm bảng 10 trường ở Bước 1 và trả lời 3 câu: 1. Hợp đồng bạn xem có công bố mã nguồn đã xác thực không? 2. Tổng cung của đồng đó là bao nhiêu? Đọc ra từ hàm nào? 3. Trong tab *Write Contract* (với USDC: *Write as Proxy*), có hàm nào cho phép một địa chỉ đặc biệt đóng băng tài khoản người khác không? Nếu có, tên hàm là gì?

Câu 3 là câu quan trọng nhất. Phần lớn sinh viên bất ngờ khi biết các đồng ổn định giá lớn đều có cơ chế đóng băng tài khoản. Đây là điểm mở cho thảo luận về mức độ phi tập trung thực tế.

Sản phẩm nộp

`forensics.md`.

LAB 4 — NHẬN DIỆN HỢP ĐỒNG CÓ RỦI RO

Buổi 4 · 75 phút · Nhóm 2 người

Gắn với công việc

Đây chính là công việc của chuyên viên thẩm định rủi ro dự án, và là kỹ năng bảo vệ chính sinh viên khỏi các dự án lừa đảo.

Chuẩn đầu ra

Sinh viên chỉ ra được điều khoản bất lợi trong mã nguồn hợp đồng và **trích dẫn được số dòng làm bằng chứng**.

Các bước

Bước 1 — Đọc thủ công trước, hỏi AI sau (30 phút)

Bắt buộc theo thứ tự này. Sinh viên đọc mã nguồn 15 phút **trước khi** được mở công cụ AI. Mục đích là để các em cảm nhận được sự khác biệt, chứ không phải để làm khó.

Bước 2 — Hỏi AI với đúng vai trò (25 phút)

Prompt mẫu (có trong `prompt_templates.md`):

Bạn là chuyên viên thẩm định rủi ro tài sản số.
Dưới đây là mã nguồn một hợp đồng token. Hãy liệt kê mọi quyền đặc biệt mà chủ sở hữu hợp đồng có thể thực hiện, và với mỗi quyền, nêu rõ:

- Tên hàm và số dòng
- Người nắm giữ token chịu rủi ro gì

Chỉ trả lời dựa trên mã nguồn tôi cung cấp. Nếu không tìm thấy, nói là không tìm thấy.

[dán mã nguồn]

Câu cuối là phần quan trọng — nó hạn chế việc AI tự bịa ra nội dung không có trong mã.

Bước 3 — Ghi nhận (20 phút)

Bảng kết luận trong `lab04.md`:

Hợp đồng	Kết luận	Tên hàm	Số dòng	Rủi ro cho người nắm giữ
A				
B				
C				

Không chấp nhận kết luận không có số dòng. Câu trả lời kiểu “AI nói hợp đồng B có rủi ro” bị tính là chưa làm.

Sản phẩm nộp

`lab04.md` + mục ghi trong `AI_JOURNAL.md` so sánh: đọc thủ công tìm ra gì, AI tìm thêm được gì, AI có nói sai chỗ nào không.

LAB 5 — VIẾT ĐẶC TẢ CHO CÔNG CỤ PHÂN TÍCH DÒNG TIỀN

Buổi 5 · 75 phút · Nhóm 2 người · KHÔNG VIẾT MÃ NGUỒN TRONG BUỔI NÀY

Gắn với công việc

Chuyên viên phân tích nghiệp vụ (BA) không viết mã, họ viết yêu cầu. Chất lượng bản yêu cầu quyết định chất lượng sản phẩm — điều này đúng với đội lập trình, và càng đúng với công cụ AI.

Chuẩn đầu ra

Một tệp `SPEC.md` đủ rõ để người khác (hoặc công cụ AI) làm ra đúng thứ mình muốn mà không cần hỏi lại.

Bài toán

Xây dựng một công cụ nhận vào một địa chỉ ví, trả về báo cáo dòng tiền vào và ra của ví đó trong 90 ngày gần nhất, kèm biểu đồ số dư theo thời gian.

Các bước

Bước 1 — Giảng viên làm mẫu một đặc tả sai và một đặc tả đúng (15 phút)

Đặc tả sai: “*Viết chương trình phân tích ví.*” → Không nói rõ ví nào, phân tích gì, kết quả ra sao. Công cụ AI sẽ tự suy đoán, và suy đoán sai.

Đặc tả đúng: xem mẫu bên dưới.

Bước 2 — Sinh viên viết `SPEC.md` (45 phút)

Bản tối thiểu phải trả lời được:

2. Đầu vào

- Một địa chỉ ví, dạng chuỗi 42 ký tự bắt đầu bằng 0x.
- Khóa API của Etherscan, đọc từ biến môi trường ETHERSCAN_API_KEY.
- Số ngày cần phân tích, mặc định 90.

3. Quy tắc nghiệp vụ

- R1: Giao dịch có trường "to" trùng địa chỉ đang xét được tính là dòng tiền vào.
- R2: Giao dịch có trường "from" trùng địa chỉ đang xét được tính là dòng tiền ra.
- R3: Với giao dịch đi ra, số tiền thực trừ khỏi ví = giá trị chuyển + phí giao dịch.
- R4: Giao dịch có trạng thái thất bại vẫn bị trừ phí, phải tính vào dòng tiền ra.
- R5: Mọi số tiền lấy về ở đơn vị wei, phải chia cho 10^{18} trước khi hiển thị.
- R6: Sắp xếp theo thời gian tăng dần.

4. Đầu ra

- Bảng dữ liệu gồm: thời gian, loại (vào/ra), số tiền ETH, phí, số dư lũy kế.
- Một biểu đồ đường: trục ngang là thời gian, trục dọc là số dư lũy kế.
- Ba con số tổng hợp: tổng vào, tổng ra, số dư cuối kỳ.

5. Trường hợp ngoại lệ

- Nếu API trả về danh sách rỗng: in thông báo "Vi không có giao dịch trong ky", không báo lỗi.
- Nếu API trả về mã lỗi: in mã lỗi và dừng, không xử lý tiếp.
- Nếu ví có hơn 10.000 giao dịch: API trả theo trang, phải lấy đủ các trang.

6. Ngoài phạm vi

- Không phân tích giao dịch token (chỉ ETH gốc).
- Không quy đổi ra tiền Việt.

Bước 3 — Kiểm tra chéo (15 phút)

Đổi `SPEC.md` cho nhóm bên cạnh. Nhóm nhận phải chỉ ra **ít nhất 2 chỗ mơ hồ**. Đây là buổi luyện kỹ năng đọc yêu cầu, có giá trị trực tiếp cho công việc BA.

Sản phẩm nộp

`SPEC.md` + nhận xét của nhóm bạn.

Lý do bắt buộc tách riêng buổi này: nếu cho phép mở công cụ AI, 100% sinh viên sẽ nhảy thẳng vào sinh mã và bỏ qua bước đặc tả. Bước đặc tả mới là phần các em bán được cho nhà tuyển dụng.

LAB 6 — SINH MÃ BẰNG AI VÀ KIỂM TRA KẾT QUẢ

Buổi 6 · 75 phút · Nhóm 2 người

Chuẩn đầu ra

Chương trình chạy ra được biểu đồ, và sinh viên ghi nhận được ít nhất 2 lỗi do công cụ AI sinh ra.

Chuẩn bị

- Đăng ký khóa API miễn phí tại `https://etherscan.io/apis`.
- Dùng Google Colab (không cần cài đặt) hoặc chạy trong Antigravity.

Các bước

Bước 1 — Giao đặc tả cho công cụ AI (15 phút)

Đọc tệp SPEC.md trong dự án và viết chương trình Python thực hiện đúng đặc tả đó.
Tuân thủ các quy ước trong AGENTS.md.
Trước khi viết mã, tóm tắt lại cách bạn hiểu yêu cầu để tôi xác nhận.

Câu cuối rất quan trọng: nó buộc công cụ nói ra cách hiểu trước khi làm, giúp phát hiện hiểu sai sớm.

Bước 2 — Chạy và kiểm tra (30 phút)

Chạy thử với địa chỉ ví mẫu do giảng viên cung cấp. **Kiểm tra 6 điểm sau — đây là danh mục kiểm tra bắt buộc:**

#	Kiểm tra	Cách kiểm	Lỗi hay gặp
1	Đơn vị tiền	Số dư hiển thị có hợp lý không?	Quên chia 10^18, hiển thị số 19 chữ số
2	Khóa API	Tìm chuỗi khóa trong mã nguồn	Ghi thẳng khóa vào mã — lỗi bảo mật nghiêm trọng
3	Phân trang	Ví có nhiều giao dịch, có lấy đủ không?	Chỉ lấy trang đầu, thiếu dữ liệu
4	Giao dịch thất bại	Có tính phí của giao dịch thất bại không?	Bỏ qua, làm sai số dư
5	Xử lý lỗi	Thử dùng khóa API sai	Chương trình dừng đột ngột, không có thông báo
6	Phiên bản API	Đối chiếu với tài liệu Etherscan hiện hành	Dùng địa chỉ endpoint đã ngừng hỗ trợ

Về điểm 6: Etherscan đã chuyển sang giao diện API hợp nhất nhiều mạng. Công cụ AI thường sinh mã theo phiên bản cũ vì dữ liệu huấn luyện của nó cũ hơn. **Đây là ví dụ điển hình nhất cho bài học của cả môn: AI biết những gì đã có, không biết những gì vừa thay đổi.** Sinh viên phải mở tài liệu chính thức để đối chiếu.

Bước 3 — Sửa và ghi nhật ký (30 phút)

Với mỗi lỗi tìm được, ghi vào `AI_JOURNAL.md` theo mẫu ở Phần B.3. Trường **“Ai phát hiện”** là trường được chấm.

15 phút cuối buổi: ba nhóm lên trình bày lỗi mình bắt được. Đây là lần đầu sinh viên được nói “công cụ AI làm sai chỗ này” và có bằng chứng. Không nên bỏ hoạt động này.

Sản phẩm nộp

Chương trình chạy được + biểu đồ + `AI_JOURNAL.md` ghi tối thiểu 2 lỗi.

LAB 7 — TÍNH CHI PHÍ VẬN HÀNH THỰC TẾ

Buổi 7 · 75 phút · Nhóm 2 người

Gắn với công việc

Đây là bài toán kinh tế, không phải bài toán kỹ thuật — và là phần sinh viên Kinh tế làm tốt hơn sinh viên CNTT. Một sản phẩm chạy được nhưng chi phí giao dịch cao hơn giá trị giao dịch thì không có mô hình kinh doanh.

Chuẩn đầu ra

Sinh viên tính được chi phí vận hành hàng tháng của một ứng dụng và kết luận về tính khả thi.

Các bước

Bước 1 — Hiểu cấu trúc phí (20 phút)

Phí một giao dịch = **Lượng gas tiêu thụ** × **Đơn giá gas**.

Loại thao tác	Lượng gas tham khảo
Chuyển ETH thông thường	21.000
Chuyển token theo chuẩn ERC-20	~50.000 – 65.000
Ghi một biến mới vào bộ nhớ lâu dài	~20.000
Sửa một biến đã có	~5.000
Triển khai một hợp đồng cỡ nhỏ	~500.000 – 1.500.000

Con số thực tế thay đổi theo mã nguồn cụ thể. Sinh viên phải **tự đo bằng Remix** ở Lab 9, không chép bảng này vào bài.

Bước 2 — Bài toán (35 phút)

Một câu lạc bộ sinh viên phát hành thẻ tích điểm trên blockchain. Mỗi tháng có 1.000 lượt cộng điểm. Mỗi lượt là một giao dịch ghi dữ liệu.

- a. Với đơn giá gas 20 Gwei và giá ETH 3.000 USD, chi phí một tháng là bao nhiêu USD?
- b. Nếu chuyển sang mạng Layer 2 với đơn giá rẻ hơn khoảng 100 lần, chi phí còn bao nhiêu?
- c. Ai trả khoản này — câu lạc bộ hay sinh viên? Nếu sinh viên trả, họ có chấp nhận không?
- d. Kết luận: mô hình này khả thi trên mạng nào?

Gợi ý cách tính: 1 Gwei = 10⁻⁹ ETH. Chi phí một giao dịch (ETH) = gas × đơn giá(Gwei) × 10⁻⁹.

Bước 3 — Mở rộng cho ý tưởng của nhóm (20 phút)

Áp dụng đúng khung tính trên cho ý tưởng đồ án nhóm đang cân nhắc.

Sản phẩm nộp

lab07.md gồm bảng tính và một đoạn kết luận về tính khả thi.

Kết luận cần rút ra: câu (d) dẫn tự nhiên đến khái niệm mạng Layer 2 — nội dung cần bổ sung vào Session 02 về mạng Lớp 2.

LAB 8 — THIẾT KẾ QUY TẮC KINH TẾ CHO SẢN PHẨM

Buổi 8 · 75 phút · Nhóm 3–4 người · BẮT ĐẦU CODEBASE NHÓM

Repo dùng từ đây đến cuối kỳ: repo nhóm. Không tạo repo mới ở Lab 9–15.

Đầu vào: repo cá nhân Lab 1–7 + danh mục 10 chủ đề ở Phần N cuối sách.

Đầu ra: repo nhóm chạy được, kế hoạch dự án, đặc tả v0.1 và quy tắc kinh tế.

Chuẩn đầu ra

Cuối buổi, người lạ mở repo phải trả lời được bốn câu: nhóm làm gì, cho ai, quy tắc chính là gì, và mỗi thành viên chịu trách nhiệm phần nào.

Các bước

Bước 1 — Lập nhóm và chọn một chủ đề (10 phút)

- Nhóm 3–4 người, ghi đủ họ tên và mã sinh viên.
- Mở Phần N, chọn đúng **một** trong 10 chủ đề.
- Viết một câu theo mẫu: “*Nhóm xây [sản phẩm] cho [người dùng] để [giải quyết vấn đề].*”
- Nếu câu dài quá hai dòng hoặc cần giải thích bằng thuật ngữ kỹ thuật, thu hẹp đề tài.

Checkpoint 1: mọi thành viên phải nói lại được câu trên mà không nhìn tài liệu.

Bước 2 — Tạo repo nhóm (15 phút)

- Một thành viên tạo repo từ `hce-web3-starter`, đặt tên ngắn, không dấu, ví dụ `hce-escrow-k58`.
- Mời các thành viên còn lại làm collaborator.
- Tạo cấu trúc thư mục đúng Phần B.6.
- Mỗi thành viên sửa một dòng trong `README.md` rồi tạo một commit; mục đích là kiểm tra tất cả đều có quyền làm việc, không phải một người giữ toàn bộ repo.

Bạn phải thấy: trang lịch sử repo có commit của tất cả thành viên.

Nếu chưa thấy: kiểm tra lời mời collaborator, đúng tài khoản GitHub và đúng nhánh `main`.

Bước 3 — Phân vai và lập kế hoạch (15 phút)

Tạo `docs/PROJECT_PLAN.md`:

```
# PROJECT PLAN — [Tên sản phẩm]
## Thành viên và vai trò
| Họ tên | Vai chính Lab 8–11 | Vai chính Lab 12–15 |

## Người dùng và vấn đề
- Người dùng chính:
- Vấn đề cần giải quyết:
- Sản phẩm cuối nhìn thấy được:

## Mốc bắt buộc
- Lab 9: contract lỗi biên dịch được
- Lab 10: audit và sửa lỗi có bằng chứng
- Lab 11: quy tắc kinh tế chạy đúng
- Lab 12: Gate Review 1
```

- Lab 13: test ca tần công/gian lận
- Lab 14: audit chéo
- Lab 15: URL DApp công khai

Bốn vai: đặc tả, hợp đồng, giao diện, kiểm thử. Nhóm 3 người có một người kiêm hai vai. Vai phải xoay ít nhất một lần trước Lab 15.

Bước 4 — Viết đặc tả v0.1 và quy tắc kinh tế (25 phút)

1. Sao chép mẫu `SPEC.md` vào `docs/SPEC.md`.
2. Viết tối thiểu 4 quy tắc có thể kiểm thử: ai được làm gì, khi nào, giới hạn bao nhiêu, lỗi thì sao.
3. Tạo `docs/ECONOMIC_RULES.md` với bốn mục: dòng tiền/quyền lợi, giới hạn chống lạm dụng, quyền quản trị, tình huống người dùng bị thiệt.
4. Dùng prompt sau để phản biện:

Bạn là người dùng thận trọng. Chỉ dựa trên SPEC và ECONOMIC_RULES dưới đây, hãy nêu 5 cách một người có thể lạm dụng quy tắc hoặc làm người khác bị thiệt. Với mỗi cách, chỉ rõ quy tắc nào chưa đủ chặt. Không viết mã.

5. Nhóm trả lời từng phản biện: sửa quy tắc hoặc ghi rõ lý do chấp nhận rủi ro.

Bước 5 — Chốt phiên bản đầu (10 phút)

Kiểm tra repo có `README.md`, `docs/PROJECT_PLAN.md`, `docs/SPEC.md`, `docs/ECONOMIC_RULES.md`, `docs/AI_JOURNAL.md`. Commit:

lab-08: khai tạo codebase nhóm và đặc tả v0.1

Sản phẩm nộp

- URL repo nhóm; ảnh lịch sử commit có đủ thành viên.
- `PROJECT_PLAN.md`, `SPEC.md`, `ECONOMIC_RULES.md`.
- Một câu giới thiệu sản phẩm và chủ đề đã chọn.

LAB 9 — HỢP ĐỒNG ĐẦU TIÊN: KẾT TIẾT KIỆM CÓ KHÓA THỜI GIAN

Buổi 9 · 75 phút · Nhóm

Repo: repo nhóm từ Lab 8.

Học kỹ thuật tại: `contracts/training/TimeLockVault.sol`.

Áp dụng vào sản phẩm tại: `contracts/project/ProjectCore.sol`.

Commit cuối buổi: `lab-09: contract loi bien dich duoc`.

Gắn với công việc

Đây là mô hình ký gửi có điều kiện — nền của mọi sản phẩm giữ hộ tiền: đặt cọc, ký quỹ mua bán, tiết kiệm có kỳ hạn.

Chuẩn đầu ra

Một hợp đồng triển khai được lên mạng Sepolia, thực hiện được trọn vẹn chu trình nạp và rút.

Bài toán

Kết tiết kiệm: người tạo kết nạp tiền vào, và chỉ rút ra được sau một mốc thời gian đã định. Trước mốc đó, kể cả người tạo cũng không rút được.

Các bước

Bước 1 — Viết `SPEC.md` (15 phút)

Tối thiểu 5 quy tắc: - R1: Ai cũng nạp được tiền vào kết. - R2: Chỉ người tạo kết mới rút được. - R3: Chỉ rút được khi thời điểm hiện tại đã qua mốc mở khóa. - R4: Số tiền nạp phải lớn hơn 0. - R5: Mọi lần nạp và rút đều phải ghi lại sự kiện để tra cứu được.

Bước 2 — Giao cho công cụ AI sinh mã (15 phút)

Viết hợp đồng Solidity theo `SPEC.md`, tuân thủ `AGENTS.md`.
Giải thích lựa chọn thiết kế trước khi đưa mã nguồn.

Bước 3 — Đối chiếu với bản mẫu (20 phút)

Giảng viên chiếu bản mẫu sau, sinh viên so sánh với bản AI sinh cho nhóm mình:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Ket tiet kiem co khoa thoi gian
contract TimeLockVault {
    address public owner;
    uint256 public unlockTime;

    event Deposited(address indexed from, uint256 amount);
    event Withdrawn(address indexed to, uint256 amount);

    error NotOwner();
    error StillLocked(uint256 unlockAt, uint256 currentTime);
    error NothingToWithdraw();
```

```

error ZeroAmount();

constructor(uint256 lockDurationSeconds) {
    owner = msg.sender;
    unlockTime = block.timestamp + lockDurationSeconds;
}

function deposit() external payable {
    if (msg.value == 0) revert ZeroAmount();
    emit Deposited(msg.sender, msg.value);
}

function withdraw() external {
    // 1. Checks – kiểm tra điều kiện
    if (msg.sender != owner) revert NotOwner();
    if (block.timestamp < unlockTime) revert StillLocked(unlockTime, block.timestamp);

    uint256 amount = address(this).balance;
    if (amount == 0) revert NothingToWithdraw();

    // 2. Effects – phát sự kiện trước khi chuyển tiền ra ngoài
    emit Withdrawn(owner, amount);

    // 3. Interactions – chuyển tiền ra ngoài sau cùng
    (bool ok, ) = payable(owner).call{value: amount}("");
    require(ok, "Chuyển tiền thất bại");
}

function timeLeft() external view returns (uint256) {
    if (block.timestamp >= unlockTime) return 0;
    return unlockTime - block.timestamp;
}
}

```

Bốn điểm cần hiểu: 1. **Thứ tự Checks – Effects – Interactions**. Kiểm tra trước, cập nhật trạng thái, chuyển tiền sau cùng. Lab 13 sẽ cho thấy đảo thứ tự này mất tiền như thế nào. 2. **error** tùy biến thay cho **require** với **chuỗi dài** — rõ hơn về phí và mô tả được dữ liệu kèm theo. 3. **call** thay cho **transfer** — **transfer** giới hạn cứng 2.300 gas, có thể thất bại khi bên nhận là hợp đồng. 4. **indexed** trong sự kiện — cho phép lọc theo địa chỉ khi tra cứu.

Bước 4 — Biên dịch, triển khai, đo phí (15 phút)

1. Mở <https://remix.ethereum.org>, dán mã nguồn.
2. Biên dịch với trình biên dịch phiên bản **0.8.20** trở lên.
3. Triển khai trên môi trường **Remix VM** trước (miễn phí, chạy trong trình duyệt). Đặt **lockDurationSeconds = 120** để chỉ phải chờ 2 phút.
4. Gọi **deposit()** với 1 ETH → thử **withdraw()** ngay → phải bị từ chối với lỗi **StillLocked**.
5. Chờ hết 2 phút → **withdraw()** → thành công.
6. **Ghi lại lượng gas** của từng thao tác (Remix hiển thị trong bảng điều khiển). Dùng cho Lab 7.
7. Chụp màn hình ba kết quả trên Remix VM và lưu vào [evidence/lab-09/](#).

Bước 5 — Chuyển kỹ thuật vào sản phẩm nhóm (10 phút)

1. Mở [docs/SPEC.md](#), khoanh **một luồng cốt lõi** của đề tài.
2. Tạo [contracts/project/ProjectCore.sol](#); chỉ cài luồng đó, chưa thêm tính năng phụ.
3. Tái sử dụng bốn ý vừa học: phân quyền, **event**, lỗi có tên và thứ tự Checks–Effects–Interactions.

4. Biên dịch. Nếu lỗi, ghi lỗi và cách sửa vào `docs/AI_JOURNAL.md`.
5. Commit đúng thông điệp ở đầu Lab.

Sản phẩm nộp

- Bảng chứng thử `TimeLockVault` và bảng gas ba thao tác trên Remix VM.
 - `ProjectCore.sol` biên dịch được, có ít nhất một luồng cốt lõi từ `SPEC.md`.
 - Liên kết commit `lab-09: contract loi bien dich duoc`.
-

LAB 10 — RÀ SOÁT MÃ NGUỒN DO AI SINH RA

Buổi 10 · 75 phút · Nhóm · BÀI QUAN TRỌNG NHẤT CỦA HỌC PHẦN

Repo: tiếp tục repo nhóm.

Bài luyện: `contracts/training/VaultBuggy.sol`.

Mục tiêu thật: tìm và sửa lỗi trong `contracts/project/ProjectCore.sol`.

Commit cuối buổi: `lab-10: audit va sua loi project core`.

Vì sao đây là bài quan trọng nhất

Mọi thứ khác trong môn học đều có thể nhờ công cụ làm hộ. Riêng việc **biết công cụ sai ở đâu** thì không. Đây cũng là thứ nhà tuyển dụng kiểm tra kỹ nhất khi phỏng vấn, kể cả với vị trí mới vào nghề.

Chuẩn đầu ra

Sinh viên tìm ra được ít nhất 3 trong 4 lỗi đã được cài sẵn, và phân biệt được lỗi nào do công cụ AI tìm ra, lỗi nào do mình tìm ra.

Các bước

Bước 1 — Đọc thủ công, không mở công cụ AI (15 phút)

Mỗi nhóm ghi ra giấy những chỗ nghi ngờ. Giảng viên đi từng bàn, không gợi ý.

Bước 2 — Chạy rà soát bằng AI (15 phút)

Bạn là kiểm toán viên hợp đồng thông minh.

Rà soát hợp đồng dưới đây và liệt kê mọi lỗ hổng, xếp theo mức nghiêm trọng.

Với mỗi lỗ hổng, nêu: dòng số mấy, khai thác thế nào, sửa ra sao.

[dán mã nguồn]

Bước 3 — Chứng minh một phát hiện bằng thực nghiệm (15 phút)

Đây là phần gây ấn tượng mạnh nhất buổi học. Triển khai hợp đồng với `_pin = 123456`, rồi đọc trực tiếp ô nhớ thứ 2 của hợp đồng:

```
// Chạy trong tab Console của trình duyệt, hoặc dùng thư viện ethers.js
await window.ethereum.request({
  method: "eth_getStorageAt",
  params: ["<địa chỉ hợp đồng>", "0x2", "latest"]
});
```

Kết quả trả về chính là giá trị `emergencyPin` dưới dạng số thập lục phân.

Kết luận cần nói rõ: từ khóa `private` trong Solidity chỉ giới hạn quyền truy cập **giữa các hợp đồng**. Nó **không** làm dữ liệu trở nên bí mật. Mọi thứ ghi lên blockchain đều công khai. Đây là hiểu lầm phổ biến nhất của người mới, và cũng là nguyên nhân của nhiều sự cố lộ dữ liệu thật.

Bước 4 — Audit contract của chính nhóm (30 phút)

1. Mở `ProjectCore.sol`; mỗi thành viên đọc một lượt, không dùng AI trong 5 phút đầu.

- 2. Dùng cùng prompt audit ở Bước 2, nhưng thêm `SPEC.md` để kiểm tra cả lỗi nghiệp vụ.
- 3. Ghi tối thiểu hai phát hiện: vị trí, hậu quả, cách tái hiện, cách sửa.
- 4. Sửa ít nhất một lỗi và biên dịch lại.
- 5. Cập nhật bảng sau trong `docs/AI_JOURNAL.md` :

Bảng bắt buộc trong `AI_JOURNAL.md` :

Lỗi	Mô tả	Ai phát hiện	Cách khắc phục
1		AI / Sinh viên	
2		AI / Sinh viên	
3		AI / Sinh viên	
4		AI / Sinh viên	

Cột “Ai phát hiện” là cột được chấm. Nhóm nào tự tìm ra lỗi 4 được cộng điểm.

Sản phẩm nộp

- Bảng phân tích `VaultBuggy.sol` và bằng chứng thực nghiệm.
- Ít nhất hai phát hiện trên `ProjectCore.sol` , trong đó có một lỗi đã sửa.
- `AI_JOURNAL.md` ghi rõ phát hiện nào do người, phát hiện nào do AI.
- Liên kết commit `lab-10: audit va sua loi project core` .

LAB 11 — CÀI QUY TẮC KINH TẾ VÀO SẢN PHẨM

Buổi 11 · 75 phút · Nhóm

Repo: tiếp tục repo nhóm.

Đầu vào: docs/ECONOMIC_RULES.md và contracts/project/ProjectCore.sol.

Bài mẫu: ClassPoint bên dưới chỉ minh họa cách biến quy tắc thành mã; không bắt mọi nhóm làm token.

Commit cuối buổi: lab-11: cài quy tắc kinh te va test.

Gắn với công việc

Chuyển từ quy tắc kinh tế trong ECONOMIC_RULES.md thành quy tắc chạy tự động. Đây là điểm gặp nhau giữa tư duy kinh tế và công cụ kỹ thuật.

Chuẩn đầu ra

ProjectCore.sol có ít nhất một quy tắc về tiền hoặc quyền lợi, kèm một ca kiểm thử thành công và một ca cố tình vi phạm quy tắc.

Các bước

Bước 1 — Đọc ví dụ chuẩn có sẵn (15 phút)

Không tự viết token từ đầu. Kế thừa thư viện OpenZeppelin — đây cũng là cách làm trong nghề: mã nguồn đã được kiểm toán nhiều lần, an toàn hơn tự viết.

Bước 2 — Bản mẫu có 2 quy tắc kinh tế (20 phút)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/// @title Diem thuong lop K58 - co thu phi va tran nam giu
contract ClassPoint is ERC20, Ownable {
    address public classFund; // vi quy lop, nhan phi
    uint256 public feeBps = 100; // 100 diem co ban = 1%
    uint256 public maxHolding; // tran nam giu moi vi

    event FeeCollected(address indexed from, uint256 amount);

    error ExceedsMaxHolding(uint256 attempted, uint256 limit);

    constructor(address _classFund)
        ERC20("K58 Class Point", "K58P")
        Ownable(msg.sender)
    {
        require(_classFund != address(0), "Vi quy khong hop le");
        classFund = _classFund;
        _mint(msg.sender, 1_000_000 * 10 ** decimals());
        maxHolding = totalSupply() * 2 / 100; // tran 2% tong cung
    }

    /// @dev OpenZeppelin phien ban 5 dung _update. Phien ban 4 dung
    /// _beforeTokenTransfer - da bi bo. Day la loi cong cu AI hay sinh ra.
```

```

function _update(address from, address to, uint256 value) internal override {
    bool isNormalTransfer = from != address(0) // không phải lúc phát hành
        && to != address(0) // không phải lúc hủy
        && from != owner(); // chủ sở hữu không bị thu phí

    if (isNormalTransfer && feeBps > 0) {
        uint256 fee = value * feeBps / 10_000;
        if (fee > 0) {
            super._update(from, classFund, fee);
            emit FeeCollected(from, fee);
            value -= fee;
        }
    }

    super._update(from, to, value);

    // Kiểm tra trần nam giữ sau khi đã cộng số dư
    if (to != address(0) && to != classFund && to != owner()) {
        if (balanceOf(to) > maxHolding) {
            revert ExceedsMaxHolding(balanceOf(to), maxHolding);
        }
    }
}
}

```

Ba điểm cần hiểu:

1. **Điểm cơ bản (basis point):** dùng số nguyên `feeBps = 100` để biểu diễn 1%, vì Solidity không có số thập phân. Đây là quy ước chuẩn trong tài chính, sinh viên Kinh tế đã quen.
2. **Vì sao có `from != owner()`** : nếu không loại trừ, chính lúc phát token cho sinh viên cũng bị thu phí. Đây là loại lỗi nghiệp vụ mà chỉ người hiểu bài toán mới phát hiện được — công cụ AI không tự biết.
3. **Ghi chú về `_update`** : OpenZeppelin phiên bản 5 đã bỏ `_beforeTokenTransfer` . Công cụ AI thường sinh mã theo phiên bản 4 và gây lỗi biên dịch. **Hãy thử tái tạo đúng lỗi này** để thấy tận mắt.

Bước 3 — Chọn và cài một quy tắc của nhóm (25 phút)

Chọn đúng **một** quy tắc trong `ECONOMIC_RULES.md` , ví dụ: phí, tiền cọc, trần nắm giữ, hạn chót, số phiếu tối thiểu, giới hạn bán lại hoặc cấm chuyển nhượng. Viết quy tắc đó vào `ProjectCore.sol` , thêm `event` và lỗi có tên. Chưa làm quy tắc thứ hai.

Bước 4 — Kiểm thử và chốt phiên bản (15 phút)

1. Chạy một ca hợp lệ: hành vi phải thành công và phát ra sự kiện.
2. Chạy một ca gian lận/vi phạm: giao dịch phải bị từ chối bằng đúng lỗi.
3. Lưu hai ảnh hoặc log vào `evidence/lab-11/` .
4. Cập nhật `SPEC.md` nếu mã và đặc tả chưa khớp; commit đúng thông điệp đầu Lab.

Mở rộng tùy chọn — Chỉ dành cho đề tài cần token

1. Trong Remix, mở tab *Solidity Compiler*, bật *Auto compile*. Thư viện OpenZeppelin được nạp tự động qua đường dẫn `@openzeppelin/...` .
2. Triển khai lên Sepolia, tham số `_classFund` là ví của một thành viên trong nhóm.
3. Trong MetaMask: *Import tokens* → dán địa chỉ hợp đồng → token hiện ra.
4. **Kiểm chứng quy tắc phí:** chuyển 1.000 token từ ví A sang ví B (ví A không phải chủ sở hữu).
 - Ví B phải nhận **990** token.
 - Ví quỹ phải tăng **10** token.

5. **Kiểm chứng tràn nắm giữ:** thử chuyển một lượng lớn khiến ví nhận vượt 2% tổng cung → giao dịch phải bị từ chối với lỗi `ExceedsMaxHolding`.

Sản phẩm nộp

- `ProjectCore.sol` có một quy tắc kinh tế khớp `ECONOMIC_RULES.md`.
- Một bằng chứng ca hợp lệ và một bằng chứng ca vi phạm bị chặn.
- Liên kết commit `lab-11: cai quy tac kinh te va test`.

Bẫy thường gặp

Bẫy	Xử lý
Lỗi biên dịch do dùng <code>_beforeTokenTransfer</code>	Đây là bài học, để sinh viên tự gặp rồi tự sửa
Quên nhân <code>10 ** decimals()</code> khi phát hành	Số lượng token sai 10^{18} lần
Phí tính ra 0 do số tiền chuyển quá nhỏ	Giải thích phép chia số nguyên làm tròn xuống

LAB 12 — GATE REVIEW 1: DUYỆT CODEBASE VÀ PHẠM VI

Buổi 12 (Tuần 4) · 75 phút · Nhóm · CÔNG DUYỆT BẮT BUỘC

Chuẩn đầu ra

Mỗi nhóm chứng minh repo từ Lab 8 đã có đặc tả, contract lỗi, quy tắc kinh tế và bằng chứng kiểm thử. Kết thúc buổi, nhóm nhận một trong ba kết luận: **Qua**, **Qua có điều kiện**, hoặc **Thu hẹp phạm vi**.

Repo: vẫn là repo nhóm, không tạo proposal/repo mới.

Tập quyết định: docs/GATE_REVIEW_1.md.

Commit cuối buổi: lab-12: gate review 1 và cập nhật phạm vi.

Quy trình Gate Review 1

Trước buổi 24 giờ

Nhóm đẩy phiên bản mới nhất lên nhánh `main`. Giảng viên chỉ xem những gì có trong repo; không nhận tệp gửi riêng qua chat hay email.

Bước 1 — Tự kiểm tra sức khỏe repo (15 phút)

- ☐ `README.md` nói rõ bài toán, người dùng và cách chạy.
- ☐ `PROJECT_PLAN.md`, `SPEC.md`, `ECONOMIC_RULES.md` còn khớp với mã.
- ☐ `ProjectCore.sol` biên dịch được.
- ☐ Có một ca hợp lệ và một ca gian lận/vi phạm bị chặn.
- ☐ Lịch sử commit có đóng góp của tất cả thành viên.

Bước 2 — Demo 3 phút/nhóm (45 phút)

- 30 giây:** ai gặp vấn đề gì.
- 30 giây:** quy tắc kinh tế/quyền lợi quan trọng nhất.
- 60 giây:** mở `ProjectCore.sol`, chạy một luồng thành công.
- 30 giây:** chạy một ca vi phạm và cho xem lỗi bị chặn.
- 30 giây:** nói việc sẽ hoàn thành tiếp theo.

Bước 3 — Nhận quyết định và thu hẹp (10 phút)

Ghi vào `docs/GATE_REVIEW_1.md`: kết luận, ba việc bắt buộc sửa, tính năng bị cắt và hạn hoàn thành. Nguyên tắc: giữ một luồng cốt lõi chạy chắc, không cố giữ nhiều tính năng nửa vời.

Bước 4 — Cập nhật kế hoạch (5 phút)

Sửa `PROJECT_PLAN.md` theo quyết định, gán người chịu trách nhiệm cho Lab 13–15 và commit.

Sản phẩm nộp

- `docs/GATE_REVIEW_1.md` có kết luận của giảng viên.
- `docs/PROJECT_PLAN.md` đã cập nhật và phạm vi sau duyệt.
- Liên kết commit `lab-12: gate review 1 và cập nhật phạm vi`.

LAB 13 — THỰC NGHIỆM MỘT VỤ MẤT TIỀN VÀ CÁCH KHẮC PHỤC

Buổi 13 · 75 phút · Nhóm

Repo: tiếp tục repo nhóm.

Bài luyện: `contracts/training/VulnerableBank.sol`.

Mục tiêu thật: viết ca kiểm thử gian lận cho `ProjectCore.sol` và vá lỗi nếu có.

Commit cuối buổi: `lab-13: them negative test va hardening`.

Gắn với công việc

Lỗi tái nhập (reentrancy) là lỗi được hỏi nhiều nhất trong phỏng vấn vị trí liên quan đến hợp đồng thông minh, kể cả vị trí mới vào nghề. Nó cũng là nguyên nhân của vụ mất 3,6 triệu ETH năm 2016 — sự cố dẫn tới việc mạng Ethereum tách làm hai.

Chuẩn đầu ra

Sinh viên tự tay rút cạn một hợp đồng có lỗi, sau đó vá lại và chứng minh cách tấn công không còn hiệu lực.

Các bước

Bước 1 — Dựng hiện trường (20 phút)

Chạy trên **Remix VM**, không dùng Sepolia (nhanh hơn và không tốn phí). 1. Triển khai `VulnerableBank`. 2. Dùng 3 tài khoản khác nhau, mỗi tài khoản gọi `deposit()` với 2 ETH. Ngân hàng có 6 ETH. 3. Gọi `bankBalance()` để xác nhận.

Bước 2 — Tấn công (15 phút)

- Triển khai `Attacker`, tham số là địa chỉ `VulnerableBank`.
- Đổi sang một tài khoản thứ tư, gọi `attack()` với 1 ETH.
- Gọi `bankBalance()` → **bằng 0**. Toàn bộ 7 ETH đã nằm trong hợp đồng tấn công.

Dừng lại ở đây 5 phút để giải thích. Vẽ lên bảng:

```
Ke tan cong goi withdraw()
  → Ngân hàng chuyển 1 ETH ← số dư vẫn còn ghi là 1 ETH
    → Ham receive() của ke tan cong tu dong chay
      → Goi withdraw() lần nữa
        → Ngân hàng lại kiểm tra số dư: vẫn còn 1 ETH → chuyển tiếp
          → ... lặp lại cho đến khi ngân hàng cạn tiền
Cuối cùng mọi chạy dòng: balances[msg.sender] = 0
```

Nguyên nhân gốc: **chuyển tiền ra ngoài trước khi cập nhật sổ sách**. Trong kế toán, đây tương đương với việc chi tiền rồi mới ghi sổ — và trong lúc chưa ghi sổ, người ta rút tiếp được.

Bước 3 — Hỏi công cụ AI theo lối đặt câu hỏi dẫn dắt (15 phút)

Bạn là kiểm toán viên hợp đồng thông minh.
Đừng đưa mã sửa ngay. Hãy giải thích từng bước điều gì xảy ra khi hàm `withdraw()` dưới đây được gọi bởi một hợp đồng có hàm `receive()`.
Sau đó nêu 2 cách khắc phục và so sánh ưu nhược điểm của từng cách.

[dán mã `VulnerableBank`]

Yêu cầu “đừng đưa mã sửa ngay” là có chủ đích — nó buộc công cụ giải thích cơ chế thay vì đưa đáp án, và buộc sinh viên phải đọc.

Bước 4 — Vá lỗi bằng hai cách (25 phút)

Cách 1 — *Đổi thứ tự (Checks – Effects – Interactions)*. Không cần thư viện, không tốn thêm phí:

```
function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0, "Khong co so du");

    balances[msg.sender] = 0;          // cap nhat so sach TRUOC

    (bool ok, ) = msg.sender.call{value: bal}("");
    require(ok, "Chuyen that bai");
}
```

Cách 2 — Dùng khóa chống gọi lại của OpenZeppelin:

```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    function withdraw() external nonReentrant { ... }
}
```

Chạy lại `attack()` → phải thất bại.

Điểm cần nhấn mạnh khi chấm vấn đáp: thứ chặn được tấn công ở cách 1 là **thứ tự các dòng lệnh**, không phải một từ khóa nào. Nhiều sinh viên sẽ trả lời “nhờ có `nonReentrant`” — câu trả lời đó cho thấy chưa hiểu bản chất.

Bước 5 — Soi lại hợp đồng của nhóm mình (10 phút)

1. Tìm mọi chỗ có chuyển tiền, gọi hợp đồng khác hoặc thay đổi quyền.
2. Viết ít nhất một ca kiểm thử thất bại trong `test/`: sai người, sai thời điểm, sai số tiền hoặc gọi lại.
3. Sửa lỗi nếu ca thử cho thấy hành vi không đúng `SPEC.md`; ghi quyết định vào `AI_JOURNAL.md`.
4. Commit đúng thông điệp ở đầu Lab.

Sản phẩm nộp

- Ảnh hoặc ảnh động chụp số dư ngân hàng trước (6 ETH) và sau khi tấn công (0 ETH).
- Bản đã vá + bằng chứng tấn công thất bại.
- Ba câu giải thích **vì sao thứ tự dòng lệnh là thứ chặn được tấn công**.
- Ca kiểm thử thất bại của `ProjectCore.sol` và commit `lab-13: them negative test va hardening`.

LAB 14 — RÀ SOÁT CHÉO GIỮA CÁC NHÓM

Buổi 14 · 75 phút · Nhóm

Đối tượng audit: `contracts/project/ProjectCore.sol` và `docs/SPEC.md` của nhóm khác.
Báo cáo: `docs/AUDIT_REPORT.md` trong repo nhóm được audit.
Commit cuối buổi: `lab-14: xu ly ket qua audit cheo`.

Gắn với công việc

Rà soát chéo là quy trình chuẩn trong mọi đội phát triển phần mềm. Với hợp đồng thông minh, nó bắt buộc, vì mã đã triển khai không sửa được.

Chuẩn đầu ra

Mỗi nhóm nộp một bản rà soát về hợp đồng của nhóm khác, có phát hiện cụ thể kèm số dòng.

Cách tổ chức

Giảng viên ghép vòng tròn: nhóm 1 rà soát nhóm 2, nhóm 2 rà soát nhóm 3, ..., nhóm cuối rà soát nhóm 1. Không cho hai nhóm rà soát lẫn nhau để tránh thỏa thuận ngầm.

Danh mục kiểm tra bắt buộc

Đưa cho sinh viên dùng như bảng chấm. Đây cũng là danh mục dùng khi tự rà soát đề án của mình.

#	Hạng mục	Câu hỏi kiểm tra
1	Phân quyền	Mọi hàm nhảy cảm có kiểm tra người gọi không? Có hàm nào quên kiểm tra không?
2	Thứ tự thao tác	Có hàm nào chuyển tiền ra ngoài trước khi cập nhật biến trạng thái không?
3	Điều kiện thời gian	Dấu so sánh có đúng chiều không? Thử nghĩ trường hợp đúng bằng mốc.
4	Phép chia	Có phép chia nào làm tròn xuống thành 0 với số nhỏ không?
5	Cách chuyển ETH	Dùng <code>call</code> hay <code>transfer</code> ? Có kiểm tra kết quả trả về không?
6	Dữ liệu riêng tư	Có dữ liệu nào tưởng là bí mật nhưng thực ra đọc được không?
7	Vòng lặp	Có vòng lặp nào chạy trên danh sách dài không giới hạn không? (có thể vượt giới hạn phí)
8	Sự kiện	Mọi thay đổi trạng thái có ghi lại sự kiện để tra cứu không?
9	Trường hợp số 0	Nạp 0 đồng, rút khi số dư 0, danh sách rỗng — xử lý thế nào?

#	Hạng mục	Câu hỏi kiểm tra
10	Địa chỉ rõ ràng	Có kiểm tra tham số địa chỉ khác <code>address (0)</code> không?

Các bước

Bước 1 — Rà soát (45 phút) — được dùng công cụ AI, nhưng mọi phát hiện phải trích số dòng.

Bước 2 — Viết báo cáo (20 phút)

```
# BÁO CÁO RÀ SOÁT – Nhóm [X] rà soát nhóm [Y]

## Tóm tắt
Đã rà soát [tên tệp], [số] dòng. Phát hiện [số] vấn đề:
[số] nghiêm trọng, [số] trung bình, [số] nhẹ.

## Phát hiện 1 – [Tên vấn đề]
- **Mức độ:** Nghiêm trọng / Trung bình / Nhẹ
- **Vị trí:** dòng [số], hàm [tên]
- **Mô tả:** ...
- **Tình huống gây thiệt hại:** Nếu [ai đó làm gì] thì [hậu quả].
- **Khuyến nghị:** ...
- **Ai phát hiện:** Công cụ AI / Thành viên nhóm
```

Bước 3 — Trao đổi và sửa lỗi (10 phút) — nhóm nhận báo cáo phản hồi từng phát hiện, sửa ngay vấn đề nghiêm trọng nhất có thể xử lý trong buổi và commit.

Quy tắc chấm

- Nhóm **bị phát hiện lỗi: không bị trừ điểm**. Mục đích là tìm ra lỗi trước khi đưa ra dùng thật.
- Nhóm **không phát hiện được gì** trong khi hợp đồng được rà soát thực sự có lỗi: **bị trừ điểm**.

Cách chấm này thưởng cho sự cẩn trọng, không thưởng cho sự dễ dãi.

Sản phẩm nộp

`docs/AUDIT_REPORT.md` + phản hồi của nhóm bị rà soát + liên kết commit sửa lỗi.

LAB 15 — GIAO DIỆN WEB VÀ ĐƯA SẢN PHẨM LÊN MẠNG

Buổi 15 · 75 phút · Nhóm · BUỔI CUỐI PHẦN NỘI DUNG

Repo: tiếp tục repo nhóm; giao diện nằm trong `web/`.
Hợp đồng kết nối: `ProjectCore.sol`, không phải hợp đồng ví dụ của Lab 9.
Commit/tag cuối buổi: `lab-15: public dapp và presentation plan`, tag `v0.1-demo`.

Chuẩn đầu ra

Một đường dẫn web công khai, mở được trên điện thoại người khác, kết nối được ví và gọi được hàm trong hợp đồng.

Nguyên tắc

Sinh viên không phải học lập trình web. Giảng viên phát sẵn một tệp HTML hoàn chỉnh. Sinh viên chỉ điền hai thông tin và chỉnh giao diện nếu muốn.

Các bước

Bước 1 — Nối giao diện với hợp đồng (20 phút) 1. Trong Remix, tab *Solidity Compiler* → nút **ABI** → sao chép. 2. Dán vào `CONTRACT_ABI`. 3. Dán địa chỉ hợp đồng đã triển khai trên Sepolia vào `CONTRACT_ADDRESS`. 4. Mở tệp bằng trình duyệt, bấm *Kết nối ví*.

Ghi bảng ánh xạ sau vào `README.md` trước khi sửa giao diện:

Thành phần	Giá trị của nhóm
Địa chỉ contract	
ABI lấy từ đâu	
Hàm đọc không tốn phí	
Hàm ghi cần xác nhận ví	

Bước 2 — Thử toàn bộ luồng (15 phút) Chạy luồng cốt lõi của chính đề tài: một thao tác thành công và một thao tác bị từ chối. Giao diện phải hiện thông báo bằng ngôn ngữ người dùng, không phải chuỗi lỗi kỹ thuật.

Phần bắt lỗi trong khối `try/catch` là chi tiết phân biệt sản phẩm dùng được và bài tập. Trường `e.shortMessage` chứa tên lỗi tùy biến đã định nghĩa trong hợp đồng — đây là lý do Lab 9 bắt dùng `error` tùy biến thay cho chuỗi `require`.

Bước 3 — Đưa lên mạng công khai (15 phút) 1. Giữ `index.html` trong `web/` của **chính repo nhóm**; không tạo repo mới. 2. Dùng GitHub Pages hoặc quy trình deploy `web/` do giảng viên cung cấp. 3. Sau vài phút có đường dẫn dạng `https://<tên tài khoản>.github.io/<tên kho>/`. 4. **Mở đường dẫn đó trên điện thoại** để kiểm tra.

Bước 4 — Chỉnh giao diện bằng công cụ AI (15 phút)

Khuyến khích, vì đây là phần sinh viên thấy sản phẩm là của mình:

Giữ nguyên toàn bộ phần JavaScript. Chỉ chỉnh phần HTML và thêm CSS để:
– Giao diện gọn, dùng được trên điện thoại

- Đổi tiêu đề thành "[tên sản phẩm của nhóm]"
- Nút bấm rõ ràng, có trạng thái đang xử lý
- Không đổi tên các thẻ id vì JavaScript đang dùng.

Câu cuối quan trọng — không có nó, công cụ AI hay đổi tên thẻ và làm hỏng phần kết nối.

Bước 5 — Lập kịch bản trình bày (10 phút)

Tạo docs/PRESENTATION_PLAN.md theo khung ở Phần C. Phân vai để tất cả thành viên đều nói và đều chỉ được một artifact do mình phụ trách. Chạy thử kịch bản 5 phút một lần.

Sản phẩm nộp

- URL công khai mở được trên điện thoại và một mã giao dịch từ DApp.
- docs/PRESENTATION_PLAN.md có thời lượng và vai của từng thành viên.
- Commit lab-15: public dapp va presentation plan và tag v0.1-demo.

Bẫy thường gặp

Bẫy	Xử lý
Ví đang ở mạng khác	Thêm kiểm tra mạng và nhắc người dùng đổi sang Sepolia
ABI dán thiếu dấu ngoặc	Kiểm tra bằng cách mở console trình duyệt xem báo lỗi
GitHub Pages chưa hiện	Chờ 2–5 phút; kiểm tra tệp có đúng tên index.html ở thư mục gốc
Mở tệp bằng đường dẫn file://	Vẫn chạy được, nhưng nên dùng đường dẫn công khai để nộp

17 PHẦN C: KẾ HOẠCH HOÀN THIỆN VÀ TRÌNH BÀY SẢN PHẨM

Dự án **không bắt đầu ở Lab 12**. Repo nhóm bắt đầu ở Lab 8, được bổ sung qua từng Lab 9–15, sau đó tiếp tục được hoàn thiện ở Buổi 16–21. Repo là nguồn duy nhất của mã, kế hoạch, bằng chứng và kịch bản trình bày.

C.1. Mốc kiểm tra

Buổi	Việc	Điều kiện đạt	Không đạt
8–11	Dựng nền repo	Đặc tả, contract lỗi, quy tắc kinh tế, ca thử	Thu hẹp luồng cốt lõi
12	Gate Review 1	Repo và phạm vi được duyệt	Bắt buộc thu hẹp
13–15	Áp bài học vào sản phẩm	Hợp đồng nhóm đã qua danh mục kiểm tra Lab 14	Nhắc lần 1
16	Hợp đồng chính	Biên dịch và chạy đúng trên Remix VM	Nhắc lần 2
17	Triển khai và kiểm thử	Có mặt trên Sepolia + tối thiểu 3 ca kiểm thử, trong đó ít nhất 1 ca thử gian lận	Nhắc lần 2

Buổi	Việc	Điều kiện đạt	Không đạt
18	Trình bày giữa kỳ	Trình diễn 5 phút, chạy thật trước lớp	Hạ mức A→B→C
19	Tổng duyệt	Áp phích + video 90 giây nộp trước 22h	Trừ 1 điểm
20	Trình diễn sản phẩm	Chạy trực tiếp trên Sepolia, không dùng video quay sẵn	Không tính điểm phần trình diễn
21	Vấn đáp cá nhân	Mỗi sinh viên giải thích một hàm được chỉ ngẫu nhiên	Áp hệ số theo LAB_CURRICULUM_ROADMAP.md mục 5.4

Vấn đáp cuốn chiếu: nhóm nào đạt mốc sớm ở Buổi 18 hoặc xong trình diễn sớm ở Buổi 20 thì vấn đáp luôn.
Mục tiêu: Buổi 21 chỉ còn dưới 20 sinh viên, nếu không sẽ không đủ thời gian.

C.2. Vai trò giảng viên trong ba tuần này

Không giảng. Đi từng bàn 10 phút, hỏi đúng ba câu: 1. Hôm nay nhóm đang vướng ở đâu? 2. Đã hỏi công cụ AI chưa, nó trả lời gì, sai chỗ nào? 3. Cho xem bản ghi thay đổi gần nhất trên GitHub.

C.3. Sản phẩm cuối mỗi nhóm mang về

- Đường dẫn web công khai, còn sống ít nhất 6 tháng.
- Kho mã nguồn GitHub có mô tả, ảnh chụp, hướng dẫn chạy, mã băm giao dịch — **dùng được khi đi phỏng vấn**.
- Áp phích khổ A2 (bài toán → giải pháp → cách hoạt động → mã QR mở sản phẩm).
- Video 90 giây quay màn hình có thuyết minh.
- Token hoặc chứng nhận của nhóm nằm lại trong ví cá nhân từng thành viên.

C.4. Kịch bản trình bày nhóm 5 phút

Thời gian	Nội dung	Bằng chứng phải mở
0:00–0:30	Người dùng và vấn đề	Câu giới thiệu trong README.md
0:30–1:15	Quy tắc kinh tế/quyền lợi chính	SPEC.md , ECONOMIC_RULES.md
1:15–2:45	Demo trực tiếp luồng cốt lõi	URL DApp và giao dịch thật
2:45–3:30	Demo một ca bị chặn	Test/log lỗi trong evidence/
3:30–4:15	Audit và quyết định an toàn	AUDIT_REPORT.md
4:15–5:00	Giới hạn, bài học, hướng tiếp theo	PRESENTATION_PLAN.md

Mọi thành viên đều phải nói và trực tiếp chỉ vào một artifact do mình phụ trách. Không dùng slide để thay cho sản phẩm; slide chỉ được dùng cho sơ đồ hoặc số liệu không thể demo trực tiếp.



PHẦN I: TẬP `prompt_templates.md` — CÁC MẪU CÂU LỆNH CHUẨN

Phát cho sinh viên từ Buổi 4. Nguyên tắc chung của mọi mẫu: **nêu vai trò** → **nêu dữ liệu** → **nêu định dạng kết quả** → **nêu điều cấm**. Câu cấm là phần sinh viên hay bỏ sót và cũng là phần quan trọng nhất.

I.1. Thẩm định hợp đồng (Lab 4, Lab 14)

Bạn là chuyên viên thẩm định rủi ro tài sản số.
Dưới đây là mã nguồn một hợp đồng token.
Hãy liệt kê mọi quyền đặc biệt mà chủ sở hữu hợp đồng có thể thực hiện.
Với mỗi quyền, nêu: tên hàm, số dòng, và rủi ro cụ thể cho người nắm giữ token.
Trình bày dưới dạng bảng.
Chỉ dựa trên mã nguồn tôi cung cấp. Nếu không tìm thấy quyền nào, hãy nói rõ là không tìm thấy, không suy đoán.

[dán mã nguồn]

I.2. Sinh mã từ đặc tả (Lab 6, Lab 9)

Đọc tệp SPEC.md trong dự án và viết mã thực hiện đúng đặc tả đó.
Tuân thủ mọi quy ước trong AGENTS.md.
Trước khi viết mã, hãy tóm tắt lại cách bạn hiểu yêu cầu để tôi xác nhận.
Nếu có chỗ nào trong đặc tả chưa rõ, hãy hỏi tôi thay vì tự quyết định.

I.3. Kiểm tra lại mã do chính AI vừa sinh (mọi lab có sinh mã)

Hãy rà soát lại đoạn mã bạn vừa viết, đóng vai một người khác đang review.
Kiểm tra cụ thể các điểm sau và trả lời từng điểm:
1. Đơn vị số liệu có đúng không (wei và ETH, phần trăm và điểm cơ bản)?
2. Có thông tin nhạy cảm nào bị ghi thẳng trong mã không?
3. Có trường hợp nào chưa xử lý: dữ liệu rỗng, số 0, người dùng không có quyền?
4. Có dùng hàm hoặc thư viện đã ngừng hỗ trợ không?
Nếu không có vấn đề ở điểm nào, nói rõ "không có vấn đề" cho điểm đó.

I.4. Rà soát an toàn theo lỗi đặt câu hỏi (Lab 13)

Bạn là kiểm toán viên hợp đồng thông minh.
Đừng đưa mã sửa ngay.
Trước hết hãy giải thích từng bước điều gì xảy ra khi hàm dưới đây được gọi bởi một hợp đồng có hàm receive().
Sau đó nêu hai cách khắc phục và so sánh ưu nhược điểm của từng cách.

[dán mã nguồn]

I.5. Sinh ca kiểm thử (Lab 14, tuần đồ án)

Dưới đây là hợp đồng của tôi.
Hãy liệt kê các ca kiểm thử cần có, chia làm ba nhóm:

- Nhóm 1: luồng bình thường
 - Nhóm 2: trường hợp biên (số 0, danh sách rỗng, đúng bằng mốc thời gian)
 - Nhóm 3: trường hợp có người cố tình gian lận
- Với nhóm 3, mô tả rõ kẻ tấn công làm gì và mong đợi hệ thống phản ứng ra sao.
Chỉ liệt kê, chưa cần viết mã kiểm thử.

I.6. Phản biện mô hình kinh tế (Lab 8)

Bạn là nhà đầu tư thận trọng đang xem xét dự án này.
Dưới đây là mô tả quy tắc kinh tế của dự án.
Hãy nêu 5 điểm yếu nghiêm trọng nhất, xếp theo mức rủi ro giảm dần.
Với mỗi điểm, nêu một tình huống cụ thể mà người dùng bị thiệt hại.
Không cần đề xuất giải pháp, tôi muốn nghe phần phê bình.

[dán nội dung ECONOMIC_RULES.md]

I.7. Chinh giao diện mà không làm hỏng phần kết nối (Lab 15)

Giữ nguyên toàn bộ phần JavaScript trong tệp này.
Chỉ chỉnh phần HTML và thêm CSS để:

- Giao diện gọn gàng, dùng được trên màn hình điện thoại
- Đổi tiêu đề thành "[tên sản phẩm]"
- Nút bấm có trạng thái đang xử lý

Không được đổi tên bất kỳ thuộc tính id nào, vì JavaScript đang dùng chúng.

I.8. Bốn câu KHÔNG nên dùng

Câu lệnh nên tránh	Vì sao
"Viết cho tôi một hợp đồng token"	Không có đặc tả. Kết quả là một hợp đồng chung chung không dùng được.
"Sửa lỗi này giúp tôi" + dán lỗi	Không có ngữ cảnh. Công cụ sẽ đoán, và thường đoán sai.
"Mã này an toàn chưa?"	Câu hỏi đóng. Công cụ có xu hướng trả lời "an toàn" cho qua. Phải hỏi "tìm lỗ hổng" thay vì "có an toàn không".
"Làm cho nó chạy được"	Công cụ sẽ bỏ qua điều kiện kiểm tra để mã chạy được — đúng nghĩa là làm hỏng phần an toàn.



PHẦN K: PHIẾU VẤN ĐÁP CÁ NHÂN (BUỔI 21)

K.1. Cách tiến hành

Mỗi sinh viên **2,5 phút**. Giảng viên mở mã nguồn đề án của nhóm, **chỉ ngẫu nhiên một hàm**, hỏi theo thứ tự cố định dưới đây. Dừng khi đủ căn cứ kết luận, không cần hỏi hết.

Vấn đáp cuốn chiếu: nhóm đặt mốc sớm ở Buổi 18 hoặc xong trình diễn sớm ở Buổi 20 thì vấn đáp luôn, để Buổi 21 chỉ còn dưới 20 sinh viên.

K.2. Bộ câu hỏi ba tầng

Tầng 1 — hiểu mã của mình (bắt buộc, hỏi mọi sinh viên)

#	Câu hỏi
1	Hàm này làm gì? Giải thích bằng lời, không đọc mã.
2	Ai được phép gọi hàm này? Chỗ nào trong mã bảo đảm điều đó?
3	Nếu người không có quyền gọi hàm này thì chuyện gì xảy ra?
4	Dòng này (chỉ vào một dòng bất kỳ) nếu xóa đi thì hỏng cái gì?

Tầng 2 — hiểu quyết định thiết kế (hỏi sinh viên trả lời tốt tầng 1)

#	Câu hỏi
5	Vì sao nhóm chọn quy tắc này mà không phải quy tắc khác? (ví dụ: vì sao phạt 10% chứ không phải 5%)
6	Chỗ nào trong hợp đồng có chuyển tiền ra ngoài? Thử tự các lệnh ở đó ra sao và vì sao?
7	Công cụ AI đã đề xuất gì mà nhóm bác bỏ? Vì sao bác bỏ?
8	Nếu người quản trị mất khóa ví thì sản phẩm này ra sao?

Tầng 3 — phản biện (hỏi sinh viên muốn đạt mức giỏi)

#	Câu hỏi
9	Nếu em là người muốn phá hệ thống này, em sẽ thử cách nào trước?
10	Quy tắc trần nắm giữ / giới hạn số lượng của nhóm có thể lách được không? Lách thế nào?
11	Sản phẩm này chạy trên mạng chính thì chi phí một tháng khoảng bao nhiêu? Ai trả?
12	Nếu phải bỏ một tính năng để kịp hạn, em bỏ tính năng nào? Vì sao?

K.3. Phiếu chấm

PHIẾU VẤN ĐÁP – Buổi 21

Sinh viên: Mã SV: Nhóm:

Hàm được hỏi:

TẦNG 1 – Hiểu mã của mình

☐ Giải thích được chức năng hàm bằng lời

☐ Chỉ đúng chỗ kiểm tra quyền

☐ Nói được hậu quả khi xóa một dòng

TẦNG 2 – Hiểu quyết định thiết kế

☐ Giải thích được lý do chọn quy tắc nghiệp vụ

- ☐ Nói đúng thứ tự lệnh khi chuyển tiền và lý do
- ☐ Kể được một đề xuất của AI đã bị bác bỏ

TẦNG 3 – Phản biện

- ☐ Nêu được một hướng tấn công hợp lý
- ☐ Nhận ra được giới hạn trong thiết kế của chính mình

HỆ SỐ LIÊM CHÍNH

- ☐ $\times 1,0$ – Trả lời trôi chảy, không cần mở tài liệu
- ☐ $\times 0,7$ – Áp ứng, phải mở mã nguồn hoặc hỏi lại công cụ AI mới trả lời được
- ☐ $\times 0,3$ – Không giải thích được logic cốt lõi trong sản phẩm mang tên mình

Điểm sản phẩm nhóm: $\times$ Hệ số: = Điểm cá nhân:

Ghi chú:

K.4. Chuẩn đánh giá hệ số

Hệ số	Biểu hiện cụ thể
$\times 1,0$	Đạt đủ tầng 1, đạt ít nhất 2/4 câu tầng 2. Trả lời bằng lời của mình, có thể sai chi tiết nhỏ nhưng đúng bản chất.
$\times 0,7$	Đạt tầng 1 nhưng phải nhìn mã nguồn để trả lời; không giải thích được lý do thiết kế ở tầng 2.
$\times 0,3$	Không trả lời được câu 1 hoặc câu 2 của tầng 1 — tức là không biết hàm mang tên mình làm gì hoặc ai được gọi nó.

Công bố hệ số này cho sinh viên ngay tại Buổi 8 và nhắc lại ở Gate Review 1, kèm ví dụ cụ thể. Công bố muộn sẽ bị khiếu nại, và đúng là khiếu nại có lý.

PHỤ LỤC: CÔNG THỨC VÀ BẢNG GAS THAM KHẢO

L.1. Cách tính nền

Phí một giao dịch (ETH) = Lượng gas $\times$ Đơn giá gas (Gwei) $\times 10^{-9}$

Phí (USD) = Phí (ETH) $\times$ Giá ETH (USD)

Giá ETH tham chiếu: dùng mức giá do giảng viên công bố tại thời điểm làm bài.

L.3. Bảng gas tham khảo để sinh viên tự ước tính

Thao tác	Gas ước tính
Chuyển ETH thông thường	21.000
Chuyển token theo chuẩn ERC-20	50.000 – 65.000

Thao tác	Gas ước tính
Ghi một biến mới vào bộ nhớ lâu dài	~20.000
Sửa một biến đã có sẵn	~5.000
Đọc dữ liệu (hàm <code>view</code>)	0 — không tốn phí
Triển khai hợp đồng cỡ nhỏ	500.000 – 1.500.000

Bắt buộc: sinh viên phải **tự đo bằng Remix** ở Lab 9 và dùng số đo của chính mình. Bảng này chỉ để ước lượng khi lập kế hoạch.

PHẦN M: HỢP ĐỒNG MẪU CHO NHÓM MỨC C

Bốn mẫu hỗ trợ **10 chủ đề ở Phần N**. Nhóm dùng mẫu như điểm khởi đầu, đổi tên biến, bỏ phần không cần và tự cài ít nhất một quy tắc kinh tế của nhóm.

M.1. Mẫu Ký quỹ — dùng cho chủ đề 1, 2, 5

Áp dụng: ký quỹ mua bán đồ cũ · đặt cọc mượn thiết bị · quỹ nhóm.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Ky quy mua ban - tien giu ho den khi nguoi mua xac nhan
contract SimpleEscrow {
    enum State { Created, Funded, Completed, Refunded }

    address public seller;
    address public buyer;
    uint256 public price;
    uint256 public deadline;      // moc tu dong cho phep hoan tien
    State public state;

    event Funded(address indexed buyer, uint256 amount);
    event Completed(address indexed seller, uint256 amount);
    event Refunded(address indexed buyer, uint256 amount);

    error WrongState();
    error NotBuyer();
    error NotSeller();
    error DeadlineNotReached();
    error WrongAmount();

    constructor(address _seller, uint256 _price, uint256 _daysToConfirm) {
        require(_seller != address(0), "Dia chi ban khong hop le");
        seller = _seller;
        price = _price;
        deadline = block.timestamp + (_daysToConfirm * 1 days);
        state = State.Created;
    }

    /// @notice Nguoi mua chuyen tien vao hop dong giu ho
```

```

function fund() external payable {
    if (state != State.Created) revert WrongState();
    if (msg.value != price) revert WrongAmount();
    buyer = msg.sender;
    state = State.Funded;
    emit Funded(msg.sender, msg.value);
}

/// @notice Người mua xác nhận đã nhận hàng, tiền về tay người bán
function confirmReceived() external {
    if (state != State.Funded) revert WrongState();
    if (msg.sender != buyer) revert NotBuyer();

    state = State.Completed;
    uint256 amount = address(this).balance;
    emit Completed(seller, amount);

    (bool ok, ) = payable(seller).call{value: amount}("");
    require(ok, "Chuyen tien that bai");
}

/// @notice Qua hạn mà người mua chưa xác nhận thì hoàn tiền
/// @dev Blockchain không tự chạy. Phải có người gọi hàm này.
function refundAfterDeadline() external {
    if (state != State.Funded) revert WrongState();
    if (block.timestamp < deadline) revert DeadlineNotReached();

    state = State.Refunded;
    uint256 amount = address(this).balance;
    emit Refunded(buyer, amount);

    (bool ok, ) = payable(buyer).call{value: amount}("");
    require(ok, "Hoan tien that bai");
}
}

```

Điểm cần lưu ý khi dùng mẫu: 1. **Máy trạng thái (enum State)** — mỗi hàm chỉ chạy ở đúng một trạng thái. Đây là cách chặn phần lớn lỗi logic, và là mẫu thiết kế dùng lại được cho hầu hết đề tài. 2. **refundAfterDeadline phải có người gọi.** Đúng nội dung slide 14 mới của Session 02 Part 2. 3. **Thứ tự Checks – Effects – Interactions** trong cả hai hàm chuyển tiền.

Gợi ý mở rộng cho nhóm muốn lên mức B: thêm cơ chế người thứ ba phân xử khi hai bên tranh chấp; thêm phí dịch vụ trích cho ban quản lý; cho phép hủy khi cả hai bên cùng đồng ý.

M.2. Mẫu Bỏ phiếu — dùng cho đề tài 3, 11

Áp dụng: bỏ phiếu bầu ban cán sự · quỹ nhóm cần nhiều người đồng ý.

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Bỏ phiếu đơn giản – mỗi vì một phiếu, kết quả không sửa được
contract SimpleVoting {
    struct Candidate { string name; uint256 voteCount; }

    address public admin;
    uint256 public startTime;
    uint256 public endTime;
}

```

```

Candidate[] public candidates;
mapping(address => bool) public isVoter;    // danh sach cu tri hop le
mapping(address => bool) public hasVoted;

event VoterRegistered(address indexed voter);
event Voted(address indexed voter, uint256 indexed candidateId);

error NotAdmin();
error NotEligible();
error AlreadyVoted();
error NotInVotingPeriod();
error InvalidCandidate();

modifier onlyAdmin() {
    if (msg.sender != admin) revert NotAdmin();
    _;
}

constructor(string[] memory names, uint256 durationMinutes) {
    admin = msg.sender;
    startTime = block.timestamp;
    endTime = block.timestamp + (durationMinutes * 1 minutes);
    for (uint256 i = 0; i < names.length; i++) {
        candidates.push(Candidate(names[i], 0));
    }
}

/// @dev Dang ky theo lo. Danh sach lop nen duoi 200 nguoi de khong vuot gioi han gas.
function registerVoters(address[] calldata voters) external onlyAdmin {
    for (uint256 i = 0; i < voters.length; i++) {
        isVoter[voters[i]] = true;
        emit VoterRegistered(voters[i]);
    }
}

function vote(uint256 candidateId) external {
    if (block.timestamp < startTime || block.timestamp > endTime) revert
NotInVotingPeriod();
    if (!isVoter[msg.sender]) revert NotEligible();
    if (hasVoted[msg.sender]) revert AlreadyVoted();
    if (candidateId >= candidates.length) revert InvalidCandidate();

    hasVoted[msg.sender] = true;           // cap nhat truoc
    candidates[candidateId].voteCount += 1;
    emit Voted(msg.sender, candidateId);
}

function getResults() external view returns (Candidate[] memory) {
    return candidates;
}

function candidateCount() external view returns (uint256) {
    return candidates.length;
}
}

```

Điểm cần lưu ý: 1. Vòng lặp trong `registerVoters` có giới hạn thực tế. Danh sách quá dài sẽ vượt giới hạn gas của một giao dịch. Phải chia lô. Đây là bài học về giới hạn tài nguyên, có trong danh mục rà soát Lab 14 mục

7. 2. **Kết quả công khai theo thời gian thực** — hàm `getResults` ai cũng gọi được, không tốn phí. Đổi lại, **bỏ phiếu ở đây không kín**: ai cũng đọc được sự kiện `Voted` để biết ai bầu cho ai. Đây là câu hỏi vấn đáp rất tốt: muốn kín thì phải làm gì, và có đáng làm không?

M.3. Mẫu Ghi nhận và xác thực — dùng cho đề tài 7, 8, 10

Áp dụng: xác thực chứng nhận khóa học · ghi nhận quyền tác giả · truy xuất nguồn gốc.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Ghi nhận và xác thực hồ sơ
/// @notice Chỉ ghi dấu vân tay số (hash) lên chuỗi, không ghi nội dung.
///         Nội dung gốc do người sở hữu tự giữ.
contract RecordRegistry {
    struct Record {
        address issuer;      // người ghi nhận
        address subject;     // người được ghi nhận
        uint256 timestamp;   // mốc thời gian, không sửa được
        string metadata;     // mô tả ngắn: tên khóa học, tên đề tài...
        bool    revoked;     // đã bị thu hồi hay chưa
    }

    address public admin;
    mapping(address => bool) public isIssuer;
    mapping(bytes32 => Record) public records;    // khóa là hash của tài liệu

    event IssuerAdded(address indexed issuer);
    event RecordCreated(bytes32 indexed docHash, address indexed subject, uint256 timestamp);
    event RecordRevoked(bytes32 indexed docHash, string reason);

    error NotAdmin();
    error NotIssuer();
    error AlreadyExists();
    error NotFound();

    constructor() {
        admin = msg.sender;
        isIssuer[msg.sender] = true;
    }

    function addIssuer(address issuer) external {
        if (msg.sender != admin) revert NotAdmin();
        isIssuer[issuer] = true;
        emit IssuerAdded(issuer);
    }

    /// @param docHash keccak256 của tệp tài liệu gốc
    function createRecord(bytes32 docHash, address subject, string calldata metadata) external {
        if (!isIssuer[msg.sender]) revert NotIssuer();
        if (records[docHash].timestamp != 0) revert AlreadyExists();

        records[docHash] = Record({
            issuer: msg.sender,
            subject: subject,
            timestamp: block.timestamp,
            metadata: metadata,
            revoked: false
        });
    }
}
```

```

    });
    emit RecordCreated(docHash, subject, block.timestamp);
  }

  function revokeRecord(bytes32 docHash, string calldata reason) external {
    Record storage r = records[docHash];
    if (r.timestamp == 0) revert NotFound();
    if (msg.sender != r.issuer && msg.sender != admin) revert NotIssuer();
    r.revoked = true;
    emit RecordRevoked(docHash, reason);
  }

  /// @notice Ai cung gọi được, không tốn phí
  function verify(bytes32 docHash)
    external view
    returns (bool exists, bool valid, address issuer, address subject, uint256 timestamp,
    string memory metadata)
  {
    Record memory r = records[docHash];
    exists = r.timestamp != 0;
    valid = exists && !r.revoked;
    return (exists, valid, r.issuer, r.subject, r.timestamp, r.metadata);
  }
}

```

Điểm quan trọng nhất khi dùng mẫu:

Chỉ ghi dấu vân tay số (hash) của tài liệu lên chuỗi, không ghi nội dung tài liệu.

Ba lý do: (1) rẻ — hash luôn 32 byte dù tài liệu bao lớn; (2) riêng tư — nội dung không bị công khai; (3) vẫn chứng minh được — ai có tài liệu gốc thì băm lại và đối chiếu, khớp là thật.

Ngược lại, sinh viên hay đề xuất ghi thẳng họ tên và điểm số lên chuỗi. Đó vừa tốn kém, vừa lộ dữ liệu cá nhân **vĩnh viễn không xóa được** — một vấn đề pháp lý thật, liên quan quy định bảo vệ dữ liệu cá nhân.

Cách tạo hash từ tệp (để sinh viên làm phần giao diện):

```

const buf = await file.arrayBuffer();
const hash = ethers.keccak256(new Uint8Array(buf));
// dùng hash này làm docHash

```

M.4. Bảng phủ đề tài

Chủ đề (Phần N)	Mẫu dùng	Việc nhóm phải tự làm thêm
1. Ký quỹ đồ cũ	M.1	Danh sách nhiều mặt hàng
2. Đặt cọc mượn thiết bị	M.1	Tính tiền phạt theo số ngày trễ
3. Quỹ nhóm nhiều người duyệt	M.2	Đề nghị chi và ngưỡng 2/3
4. Giải ngân học bổng	M.2 + M.3	Vai trò nhà tài trợ, đơn vị xác nhận, người nhận
5. Gây quỹ hoàn tiền	M.1	Nhiều người góp, mỗi người rút phần của mình

Chủ đề (Phần N)	Mẫu dùng	Việc nhóm phải tự làm thêm
6. Vé sự kiện	M.5	Trần giá bán lại và check-in một lần
7. Huy hiệu không chuyển nhượng	M.5	Đặt <code>soulbound = true</code> , giao diện tra cứu
8. Xác thực chứng nhận	M.3	Giao diện tra cứu và thu hồi
9. Dấu thời gian nghiên cứu	M.3	Tạo hash từ tệp phía giao diện
10. Truy xuất đặc sản Huế	M.3	Nhiều chặng và phân quyền theo vai

M.5. Mẫu Vé / Chứng nhận theo chuẩn ERC-721 — dùng cho chủ đề 6 và 7

Áp dụng: vé sự kiện · chứng nhận không chuyển nhượng.

Mỗi vé hay chứng nhận là **một vật riêng biệt, có số hiệu riêng** — khác với thẻ tích điểm ở Lab 11, nơi mọi điểm đều như nhau. Đó là lý do dùng chuẩn ERC-721 thay vì ERC-20.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/// @title Vé sự kiện / Chứng nhận
/// @notice Bat công tác soulbound để chuyển thành chứng nhận không chuyển nhượng được
contract EventTicket is ERC721, Ownable {
    uint256 public nextTokenId = 1;
    uint256 public maxSupply;
    uint256 public price;
    bool    public soulbound;                // true = cam chuyển nhượng

    mapping(uint256 => bool) public used;      // vé đã check-in chưa
    mapping(address => bool) public hasTicket; // mỗi ví chỉ mua được một vé

    event TicketMinted(address indexed to, uint256 indexed tokenId);
    event TicketUsed(uint256 indexed tokenId, address indexed by);

    error SoldOut();
    error WrongPrice();
    error AlreadyOwns();
    error AlreadyUsed();
    error NotTransferable();
    error NotFound();

    constructor(string memory name_, string memory symbol_, uint256 _maxSupply, uint256 _price,
        bool _soulbound)
        ERC721(name_, symbol_)
        Ownable(msg.sender)
    {
        maxSupply = _maxSupply;
        price = _price;
        soulbound = _soulbound;
    }

    /// @notice Người tham dự tu mua vé
    function buy() external payable {
```

```

        if (nextTokenId > maxSupply) revert SoldOut();
        if (msg.value != price) revert WrongPrice();
        if (hasTicket[msg.sender]) revert AlreadyOwns();

        uint256 id = nextTokenId;
        nextTokenId++; // cap nhat truoc khi mint
        hasTicket[msg.sender] = true;

        _safeMint(msg.sender, id);
        emit TicketMinted(msg.sender, id);
    }

    /// @notice Ban to chuc cap ve mien phi (dung cho chung nhan)
    function issueTo(address to) external onlyOwner {
        if (nextTokenId > maxSupply) revert SoldOut();
        uint256 id = nextTokenId;
        nextTokenId++;
        _safeMint(to, id);
        emit TicketMinted(to, id);
    }

    /// @notice Soat ve tai cua. Chi ban to chuc goi duoc.
    function checkIn(uint256 tokenId) external onlyOwner {
        if (_ownerOf(tokenId) == address(0)) revert NotFound();
        if (used[tokenId]) revert AlreadyUsed();
        used[tokenId] = true;
        emit TicketUsed(tokenId, _ownerOf(tokenId));
    }

    /// @dev OpenZeppelin v5: moi thay doi chu so huu deu di qua _update.
    ///      Mint co from == address(0), burn co to == address(0) – phai chua hai truong hop
    nay.
    function _update(address to, uint256 tokenId, address auth)
        internal override returns (address)
    {
        address from = _ownerOf(tokenId);
        bool isTransfer = from != address(0) && to != address(0);

        if (soulbound && isTransfer) revert NotTransferable();
        if (isTransfer && used[tokenId]) revert AlreadyUsed(); // ve da soat thi khong ban lai
        duoc

        return super._update(to, tokenId, auth);
    }

    function withdraw() external onlyOwner {
        uint256 amount = address(this).balance;
        (bool ok, ) = payable(owner()).call{value: amount}("");
        require(ok, "Rut tien that bai");
    }
}

```

Ba điểm cần kiểm tra khi dùng mẫu này:

#	Nội dung	Vì sao quan trọng
1	<code>_update</code> là điểm chặn duy nhất trong OpenZeppelin v5	Công cụ AI rất hay sinh ra <code>_beforeTokenTransfer</code> — đó là cách của phiên bản 4, biên dịch không nổi. Đây chính là lỗi đã dạy nhận diện ở Lab 10.

#	Nội dung	Vì sao quan trọng
2	Phải loại trừ mint và burn	Nếu chỉ viết <code>if (soulbound) revert</code> thì cả hàm phát hành cũng hỏng — không cấp nổi một chứng nhận nào. Đây là lỗi nhóm mức C mắc nhiều nhất.
3	<code>soulbound</code> quyết định bản chất sản phẩm	<code>false</code> → vé sự kiện, chuyển nhượng được, có thị trường thứ cấp. <code>true</code> → chứng nhận, gắn với người, không mua bán được. Một biến, hai đề tài.

Câu hỏi vấn đáp gợi ý: “Vé đã soát rồi thì hợp đồng chặn chuyển nhượng. Vì sao? Nếu bỏ dòng đó thì ai lợi, ai thiệt?”

Gợi ý mở rộng lên mức B: thêm hàm hoàn vé trước hạn (trả lại tiền và đốt vé); thêm hạng vé với giá khác nhau; thêm `tokenURI` trỏ tới ảnh vé.

Lưu ý triển khai trên Remix: dòng `import "@openzeppelin/..."` chạy thẳng được trên Remix nhờ cơ chế tự tải thư viện. Nếu Remix báo không tìm thấy, đổi sang đường dẫn đầy đủ `https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v5.0.0/contracts/token/ERC721/ERC721.sol`.



PHẦN N: 10 CHỦ ĐỀ DỰ ÁN PHÙ HỢP SINH VIÊN NĂM 3

Chọn một chủ đề ở Lab 8. Mục tiêu là một luồng cốt lõi chạy được, không phải mô phỏng cả một doanh nghiệp. Nhóm chưa tự tin nên chọn chủ đề có nhãn **Dễ**.

#	Chủ đề	Người dùng và bài toán	Luồng cốt lõi phải demo	Mức
1	Ký quỹ mua bán đồ cũ KTX	Sinh viên sợ giao tiền trước mà không nhận hàng	Người mua nạp cọc → xác nhận nhận hàng → người bán nhận tiền; quá hạn được hoàn	Dễ
2	Đặt cọc mượn thiết bị	Câu lạc bộ quản lý máy chiếu, máy ảnh	Nhận thiết bị → khóa cọc → trả đúng hạn/khấu trừ trễ hạn	Dễ
3	Quỹ nhóm nhiều người duyệt	Nhóm sinh viên muốn chi quỹ minh bạch	Tạo đề nghị chi → đủ 2/3 phê duyệt → thực hiện một lần	Vừa
4	Giải ngân học bổng minh bạch	Nhà tài trợ và sinh viên cần thấy đúng điều kiện giải ngân	Tạo suất → xác nhận mốc → giải ngân; sai người/sai mốc bị chặn	Vừa
5	Gây quỹ có hoàn tiền	Dự án sinh viên cần góp vốn theo mục tiêu	Góp trước hạn → đủ mục tiêu thì rút; không đủ thì từng người hoàn phần của mình	Vừa
6	Vé sự kiện giới hạn bán lại	Ban tổ chức muốn chống vé giả và đầu cơ	Mua vé → chuyển nhượng trong trần giá → check-in một lần	Khó

#	Chủ đề	Người dùng và bài toán	Luồng cốt lõi phải demo	Mức
7	Huy hiệu thành tích không chuyển nhượng	CLB/nhà trường cấp ghi nhận không thể mua bán	Đơn vị có quyền cấp → ví nhận huy hiệu → chuyển nhượng bị chặn	Vừa
8	Xác thực chứng nhận khóa học	Nhà tuyển dụng cần kiểm tra chứng nhận	Đơn vị phát hành ghi hash → bên thứ ba tra cứu → chứng nhận bị thu hồi hiện rõ	Dễ
9	Dấu thời gian ý tưởng nghiên cứu	Sinh viên cần chứng minh tệp đã tồn tại tại một thời điểm	Bấm tệp trên web → ghi hash → tệp khác không xác thực được	Dễ
10	Truy xuất đặc sản Huế	Cơ sở và khách mua cần lịch sử lô hàng	Tạo lô → thêm chặng bởi đúng vai → quét QR xem lịch sử	Vừa

Bộ lọc duyệt phạm vi

Đề tài chỉ được duyệt khi trả lời **có** cho cả bốn câu: (1) người lạ mở sản phẩm và hiểu trong 10 giây; (2) giải thích được trong một câu; (3) `ProjectCore.sol` dưới 150 dòng; (4) có ít nhất một quy tắc về tiền, quyền hoặc chống lạm dụng mà bằng tính thông thường không giải quyết đủ.

Giới hạn chung: hợp đồng thông minh không tự thức dậy theo thời gian. Đến hạn vẫn phải có người gọi hàm; hợp đồng chỉ kiểm tra thời điểm rồi cho phép hoặc từ chối.